



UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Análise Comparativa de LLMs Open-Source para a Conversão de Normas de Arquitetura, Engenharia e Construção em Representações RASE

Proposta de Dissertação de Mestrado

Eike Natan Sousa Brito



São Cristóvão – Sergipe

2026

UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Eike Natan Sousa Brito

**Análise Comparativa de LLMs Open-Source para a Conversão
de Normas de Arquitetura, Engenharia e Construção em
Representações RASE**

Proposta de Dissertação de Mestrado apresentada ao Programa de Pós-Graduação em Ciência da Computação da Universidade Federal de Sergipe como requisito parcial para a obtenção do título de mestre em Ciência da Computação.

Orientador(a): Andre Britto de Carvalho

São Cristóvão – Sergipe

2026

Resumo

A verificação automática de conformidade de normas técnicas no setor de Arquitetura, Engenharia e Construção (AEC) ainda é limitada pela linguagem técnica e pela falta de padronização dos documentos normativos. A metodologia RASE (*Requirement, Applicability, Selection, Exception*) converte normas em dados computáveis, e Modelos de Linguagem de Grande Porte (LLMs) têm sido investigados para apoiar essa conversão. Esta pesquisa avalia comparativamente seis LLMs *open-source* — Llama, Dolphin, Gemma, Mistral, Alpaca e Qwen — na conversão automática de normas brasileiras da AEC para o formato RASE, utilizando **exclusivamente Engenharia de Prompt**. A tarefa é decomposta em três níveis (N1: segmentação atômica; N2: identificação dos operadores RASE; N3: estruturação semântica em JSON) e avaliada em cinco experimentos (EN1, EN2, EN1N2, EN3 e EN1N2N3) sobre um *dataset* de 79 normas anotadas baseado na NBR 9050. A qualidade foi medida por métricas de similaridade lexical, semântica e de distância, complementadas por métricas de classificação. O Llama lidera a qualidade global, com similaridade semântica superior a 0,80 em EN2, enquanto o Dolphin apresenta o melhor custo-benefício ao combinar qualidade competitiva com tempo de execução até cem vezes menor. As métricas semânticas mostraram-se mais adequadas que as lexicais, e o pipeline encadeado evidenciou a propagação esperada de erros entre os níveis.

Palavras-chave: Aprendizado de Máquina, Processamento de Linguagem Natural, LLM, Engenharia de Prompt, BIM, RASE.

Abstract

The adoption of Building Information Modeling (BIM) has been digitally transforming the construction industry by centralizing information throughout the entire project lifecycle, increasing accuracy, reducing errors, and optimizing resources. This growth, however, brings challenges related to the management of large volumes of data and, above all, to the integration of complex regulatory standards. In the Architecture, Engineering, and Construction (AEC) sector, automatic compliance checking is still hindered by technical language and the lack of standardization of normative documents. A promising alternative is the RASE methodology (*Requirement, Applicability, Selection, Exception*), which converts standards into computable data. In parallel, Large Language Models (LLMs) have been investigated to interpret regulatory texts, but systematic evaluations of open-source LLMs applied to Brazilian standards are still scarce. This research comparatively evaluates six open-source LLMs — Llama, Dolphin, Gemma, Mistral, Alpaca, and Qwen — for the automatic conversion of AEC technical standards into the RASE format, using **exclusively Prompt Engineering** (without Fine-Tuning and without Retrieval-Augmented Generation). The task is decomposed into three successive normalization levels: N1 — atomic segmentation of normative sentences; N2 — identification of the RASE operators (Requirement, Applicability, Selection, Exception); and N3 — semantic structuring into JSON. Three experiments were conducted: EN1 (N1 in isolation), EN2 (N2 in isolation, from the reference N1), and EN1N2 (chained pipeline), allowing the analysis of error propagation between stages. The models were served locally via Ollama and applied to a dataset of 79 annotated Brazilian standards, based on NBR 9050. Output quality was assessed by lexical similarity metrics (FuzzyWuzzy, TF-IDF), semantic similarity metrics (SBERT, BERTimbau, Multilingual), and distance metrics (Word Mover's Distance with FastText and NILC embeddings), complemented by classification metrics (Accuracy, Precision, Recall, and F1-score). Results indicate that Llama leads in overall quality, achieving semantic similarity above 0.80 in the EN2 experiment, while Dolphin shows the best cost-benefit by combining competitive quality with execution time up to one hundred times lower. Semantic metrics proved to be more suitable than lexical ones for evaluating normative transformations, and the chained pipeline highlighted the expected propagation of errors between levels. The research contributes with the operational definition of the three normalization levels over the RASE methodology, with a reproducible set of prompts specialized by level and operator, and with the first systematic comparison of six open-source LLMs for the conversion of Brazilian AEC standards.

Keywords: Machine Learning, Natural Language Processing, LLM, Prompt Engineering, BIM, RASE.

Lista de ilustrações

Figura 1 – Representação do BIM - Fonte: HOCH	16
Figura 2 – Matriz de confusão.	18
Figura 3 – Exemplo da transformação de palavras em <i>word embeddings</i>	22
Figura 4 – Exemplo da transformação de <i>word embeddings</i> para <i>sentence embeddings</i> . .	23
Figura 5 – Rede Neural MLP.	25
Figura 6 – Estrutura de um neurônio no MLP.	26
Figura 7 – Exemplo do deslocamento do gráfico através da mudança da constante bias. .	26
Figura 8 – Tipos de camadas e conexões de nodos.	30
Figura 9 – Exemplo de um modelo seq2seq para tradução do inglês para português. . .	33
Figura 10 – Exemplo de um modelo com o mecanismo de atenção para tradução do inglês para português.	35
Figura 11 – Exemplo do cálculo de uma camada de <i>self-attention</i>	36
Figura 12 – Representação da pilha (<i>stack</i>) do <i>transformer</i>	37
Figura 13 – Camadas do <i>encoder</i> e <i>decoder</i> do <i>transformer</i>	38
Figura 14 – Utilização da metodologia RASE na NBR 9050	44

Lista de abreviaturas e siglas

DCOMP	Departamento de Computação
UFS	Universidade Federal de Sergipe
BIM	Building Information Modeling ou Modelagem da Informação da Construção
AEC	Architecture, Engineering and Construction ou Arquitetura, Engenharia e Construção
IA	Artificial Intelligence ou Inteligência Artificial
IAG	Generative AI ou IA Generativa
AM	Machine Learning ou Aprendizado de Máquina
PLN	Natural Language Processing ou Processamento de Linguagem Natural
RNA	Artificial Neural Networks ou Redes Neurais Artificiais
MLP	Multi-Layer Perceptron ou Perceptron Multi-Camadas
RNN	Recurrent Neural Networks ou Redes Neurais Recorrentes
LSTM	Long Short-Term Memory ou Memória de Longo e Curto Prazo
BiLSTM	Bidirectional Long Short-Term Memory ou Memória de Longo e Curto Prazo Bidirecional
LLM	Large Language Model ou Modelo de Linguagem de Grande Porte
EP	Prompt Engineering ou Engenharia de Prompt
BERT	Bidirectional Encoder Representations from Transformers
SBERT	Sentence-BERT
TF-IDF	Term Frequency–Inverse Document Frequency
WMD	Word Mover’s Distance ou Distância do Movimentador de Palavras
JSON	JavaScript Object Notation
NBR	Norma Brasileira
RASE	Requirement, Applicability, Selection, Exception

N1	Nível 1 de normalização — Segmentação Atômica
N2	Nível 2 de normalização — Identificação dos Operadores RASE
N3	Nível 3 de normalização — Estruturação Semântica em JSON
EN1	Experimento de avaliação isolada do nível N1
EN2	Experimento de avaliação isolada do nível N2
EN1N2	Experimento do pipeline encadeado $N1 \rightarrow N2$
ACC	Accuracy ou Acurácia
PRE	Precision ou Precisão
REC	Recall ou Revocação
F1	F1-score ou Medida F1
MSE	Mean Squared Error ou Erro Quadrático Médio
RMSE	Root Mean Squared Error ou Raiz do Erro Quadrático Médio
MAE	Mean Absolute Error ou Erro Médio Absoluto

Lista de símbolos

θ	Bias
w	Peso
u	Ponto de ativação
q	Query
k	Key
v	Value
W^q	Cabeça de atenção da Query
Q^k	Cabeça de atenção da Key
W^v	Cabeça de atenção do Value

Sumário

1	Introdução	10
1.1	Objetivos	12
1.2	Organização do Documento	13
2	Fundamentação Teórica	14
2.1	BIM	14
2.2	Aprendizado de Máquina	16
2.3	PLN	19
2.4	Métricas de Similaridade Textual	22
2.5	Redes Neurais Artificiais	24
2.6	Memória Longa de Curto Prazo	31
2.7	Transformers	32
2.8	LLM	38
2.9	RASE	42
2.10	Trabalhos Relacionados	45
2.10.1	Análise dos Trabalhos Relacionados	46
3	Metodologia	48
3.1	Visão Geral	48
3.2	Necessidade de Normalização em Três Níveis	48
3.2.1	N1 — Segmentação Atômica	49
3.2.2	N2 — Identificação dos Operadores RASE	49
3.2.3	N3 — Estruturação Semântica em JSON	49
3.3	Engenharia de Prompt	50
3.3.1	Prompt Direct	50
3.3.2	Prompt Chain-of-Thought	50
3.3.3	Prompt Few-Shot	50
3.4	Modelos LLM Avaliados	50
3.5	Dataset	51
3.6	Pipeline de Execução e Experimentos	51
3.6.1	Experimento EN1	52
3.6.2	Experimento EN2	52
3.6.3	Experimento EN1N2	52
3.6.4	Experimento EN3	52
3.6.5	Experimento EN1N2N3	52
3.7	Métricas de Validação	52

3.7.1	Similaridade Lexical	53
3.7.2	Similaridade Semântica	53
3.7.3	Distância Semântica (WMD)	53
3.7.4	Métricas de Classificação	53
3.8	Ambiente Computacional	54
4	Resultados	55
4.1	Visão Geral dos Experimentos	55
4.2	Resultados por Experimento	56
4.2.1	EN1 — Segmentação N1	56
4.2.2	EN2 — Identificação dos Operadores RASE	56
4.2.3	EN1N2 — Pipeline Encadeado N1 e N2	57
4.2.4	EN3 — Estruturação Semântica em JSON	57
4.2.5	EN1N2N3 — Pipeline Encadeado Completo	58
4.3	Resultados por Modelo	58
4.4	Análise por Métrica	59
4.5	Tempo de Execução	60
4.6	Discussão Integrada	61
5	Conclusão	62
5.1	Síntese dos Resultados	62
5.2	Contribuições	62
5.3	Limitações	63
5.4	Trabalhos Futuros	63
	Referências	65

1

Introdução

A área de execução de projetos e fiscalização de obras sustenta-se cada vez mais por meio de tecnologias e metodologias que asseguram uma melhor aplicação dos recursos. Com a evolução tecnológica, tornou-se possível corrigir deficiências do setor e atender às exigências do mercado. Nesse contexto, muitas empresas de engenharia estão migrando de padrões bidimensionais (2D) para tridimensionais (3D). Essa transição introduz o conceito de *Building Information Modeling* (BIM), uma metodologia inovadora para os setores de Arquitetura, Engenharia e Construção (AEC) (EASTMAN; TEICHOLZ; SACKS, 2011).

O BIM (*Building Information Modeling*) é um processo que abrange o planejamento e a execução de construções à luz de modelos tridimensionais detalhados. A metodologia pode ser classificada em diversas dimensões, de 3D a 7D, cada uma representando um aspecto específico da utilização do BIM: produção do modelo ou projetos (3D); planejamento de tempo (4D); gestão de custos (5D); sustentabilidade do edifício (6D); e gerenciamento, manutenção e operação pós-construção (7D) (EASTMAN; TEICHOLZ; SACKS, 2011). É importante ressaltar que o BIM não é um software, mas sim um conceito fundamentado em sistemas informatizados como o CAD. Alguns dos softwares que implementam o conceito BIM são o Revit®, da Autodesk®, e o ArchiCAD®, da Graphisoft®.

O BIM permite criar modelos virtuais detalhados, facilitando a visualização e a detecção de interferências antes da construção física. As principais aplicações do BIM incluem análise de projetos, orçamento e planejamento, coordenação modular, gestão do ciclo de vida, visualização arquitetônica, entre outros (VARGAS, 2023a).

Com a adoção do BIM, a quantidade de dados gerados por projetos cresce significativamente, atraindo o interesse de cientistas de dados. Esses profissionais trabalham com grandes volumes de informações, coletando, analisando e interpretando dados para gerar lucros para as organizações (DAVENPORT; PATIL, 2012). Uma das áreas mais promissoras para cientistas de dados é o Aprendizado de Máquina (AM).

O Aprendizado de Máquina, uma subárea da Inteligência Artificial (IA), explora a criação de algoritmos capazes de compreender dados de forma autônoma (TAN; STEINBACH; KUMAR, 2006). Por meio do AM, é possível extrair e reconhecer padrões em grandes volumes de dados, construindo modelos que executam tarefas complexas, fazem previsões e reagem de maneira inteligente a diferentes situações.

No campo da Engenharia Civil, a IA tem transformado processos tradicionais em abordagens automatizadas, promovendo maior eficiência, precisão e sustentabilidade. Algoritmos de aprendizado de máquina, redes neurais e aprendizado profundo permitem que sistemas revisem e otimizem projetos desde a fase de planejamento até a execução de grandes obras (FUCHS et al., 2024).

O trabalho de Liu investigou o uso de Aprendizado de Máquina para melhorar o processo de montagem na construção por meio de regras de design. O autor concluiu que a combinação de BIM, AM e regras de design pode otimizar o processo de montagem, resultando em projetos mais eficientes e econômicos (LIU; ZHANG; LI, 2022). O autor Alawadhi treinou Redes Neurais Artificiais com dados BIM com o objetivo de reconhecer objetos de construção em fotos e concluiu que as imagens geradas por computador dos modelos BIM podem resultar em segmentação semântica eficaz em fotos reais de edifícios, indicando a viabilidade de usar dados sintéticos para treinar IA em problemas arquitetônicos do mundo real (ALAWADHI; YAN, 2023). O cientista Saka realizou um treinamento com dados BIM para prever a circularidade de resíduos de demolição de construção. A partir de 2.280 projetos de demolição, foi possível desenvolver um sistema capaz de prever as quantidades de materiais recicláveis e destinar os resíduos para aterros de demolição, facilitando um melhor planejamento e gestão nessa área (SAKA et al., 2024).

No entanto, o BIM não oferece uma solução direta para a gestão de normas regulatórias voltadas para o setor AEC, uma vez que essas normas são reguladas por região. Cada região possui suas próprias regras, muitas das quais são interpretativas, dificultando a automação desse processo (VARGAS, 2023b).

O Processamento de Linguagem Natural (PLN) possibilita que modelos de AM reconheçam padrões e processem informações, gerando resultados como sistemas de tradução, conversas e respostas automatizadas. Uma subárea emergente do PLN é a IA Generativa (IAG), cujos modelos são capazes de criar novos dados com base em seus *datasets* de treinamento. Esses modelos são amplamente usados na geração de textos, imagens e áudios. Exemplos de modelos generativos incluem o ChatGPT, da OpenAI (OPENAI et al., 2024), e o Llama 3, da Meta (AI@META, 2024). Esses algoritmos são projetados para produzir respostas contextualmente sensíveis em linguagem humana e são classificados como LLMs (INAN et al., 2023).

Os Modelos de Linguagem de Grande Escala (LLMs) são sistemas de inteligência artificial projetados para compreender e gerar linguagem humana de forma avançada. Esses modelos são treinados em vastas quantidades de dados textuais, permitindo-lhes capturar nuances linguísticas e contextuais. Arquiteturas como os transformadores são comumente utilizadas

nesses modelos devido à sua eficácia em processar sequências de texto (ZHAO et al., 2023).

Tecnologias relacionadas a LLMs têm transformado o acesso ao conhecimento e a interação com o mundo. Dentre as estratégias para adaptar esses modelos a domínios específicos, a *Engenharia de Prompt* (EP) destaca-se como a abordagem mais leve e acessível, por dispensar reescalonamento de pesos e infraestrutura adicional de recuperação.

No setor de Arquitetura, Engenharia e Construção (AEC), os Modelos de Linguagem de Grande Escala (LLMs) podem auxiliar na interpretação de normas regulatórias e na conversão dessas informações em formatos processáveis por máquinas (DU; NOUSIAS; BORRMANN, 2024). Contudo, há lacunas na literatura quanto à integração da LLMs com tecnologias emergentes na indústria AEC. Uma dessas lacunas está na interpretação de leis, códigos e padrões regulatórios em níveis nacionais e internacionais, que historicamente apresentam desafios para a IA devido à complexidade semântica desses documentos (FUCHS et al., 2024).

Para abordar esse problema, Hjelseth desenvolveu a metodologia RASE, que transforma normas regulatórias em dados computáveis (HJELSETH; NISBET, 2011). Os dados computáveis na metodologia RASE são informações extraídas de normas regulatórias e estruturadas em quatro atributos: **Requisito**, que define o que deve ser atendido; **Aplicabilidade**, que indica onde a regra se aplica; **Seleção**, que apresenta opções para cumpri-la; e **Exceção**, que especifica casos em que a regra não se aplica. Essa organização permite que softwares BIM interpretem e verifiquem automaticamente a conformidade de projetos, facilitando ajustes e validações com maior eficiência e precisão (HJELSETH; NISBET, 2011).

Com a transformação das normas regulatórias para o formato RASE, torna-se possível desenvolver um software integrado ao Revit, capaz de validar cada elemento do projeto de acordo com as normas apresentadas (SANTOS; SAMPAIO, 2021). Contudo, a transformação das normas para o formato RASE é atualmente realizada de forma manual, exigindo que cada regra seja analisada e convertida individualmente. Além disso, dependendo do projeto BIM, pode haver centenas ou até milhares de elementos, o que gera um alto custo de tempo para realizar todo o processo de conversão e validação dos dados.

1.1 Objetivos

Objetivo geral

Avaliar comparativamente o uso de seis Modelos de Linguagem de Grande Porte (LLMs) *open-source* (Llama, Dolphin, Gemma, Mistral, Alpaca e Qwen) na conversão automática de normas técnicas brasileiras da AEC (em especial a NBR 9050) para o formato RASE, utilizando exclusivamente Engenharia de Prompt em três níveis sucessivos de normalização (N1, N2 e N3).

Objetivos específicos

- Definir e implementar três níveis de normalização sobre a metodologia RASE: **N1** (segmentação atômica de sentenças normativas); **N2** (identificação dos operadores RASE (Requisito, Aplicabilidade, Seleção e Exceção)); e **N3** (estruturação semântica das regras em formato JSON).
- Projetar prompts adequados a cada nível, empregando as técnicas *Prompt Direct*, *Prompt Chain-of-Thought* e *Prompt Few-Shot*.
- Comparar os seis LLMs *open-source* sob cinco experimentos: **EN1** (avaliação isolada do N1), **EN2** (avaliação isolada do N2 com o N1 de referência como entrada), **EN1N2** (pipeline encadeado $N1 \rightarrow N2$), **EN3** (avaliação isolada do N3 com o N2 de referência como entrada) e **EN1N2N3** (pipeline encadeado completo $N1 \rightarrow N2 \rightarrow N3$).
- Avaliar a qualidade das saídas com métricas de similaridade lexical (FuzzyWuzzy, TF-IDF), semântica (SBERT, BERTimbau, Multilingual) e de distância (*Word Mover's Distance* com FastText e NILC), complementadas por métricas de classificação (Acurácia, Precisão, Revocação e F1-score).
- Analisar a propagação de erros entre as etapas do pipeline e o custo computacional (tempo) por modelo.

1.2 Organização do Documento

O restante deste documento está estruturado da seguinte forma. O **Capítulo 2 — Fundamentação Teórica** apresenta os conceitos de BIM, Aprendizado de Máquina, Processamento de Linguagem Natural, Redes Neurais (incluindo LSTM, BiLSTM e Transformers), Modelos de Linguagem de Grande Porte e a metodologia RASE, além das métricas utilizadas e dos trabalhos relacionados. O **Capítulo 3 — Metodologia** descreve os níveis de normalização N1, N2 e N3, as técnicas de Engenharia de Prompt aplicadas, os seis modelos LLM avaliados, o *dataset* de 79 normas, o pipeline de execução e as métricas de validação. O **Capítulo 4 — Resultados** apresenta o desempenho por modelo e por métrica nos experimentos EN1, EN2, EN1N2, EN3 e EN1N2N3, os tempos de execução observados e a análise comparativa. O **Capítulo 5 — Conclusão** sintetiza as contribuições, discute as limitações e aponta direções para trabalhos futuros.

2

Fundamentação Teórica

2.1 BIM

O BIM (*Building Information Modeling*, ou Modelagem da Informação da Construção) não é uma tecnologia específica, mas sim um processo integrado ao planejamento e execução de construções. De acordo com [Penttilä \(2016\)](#), o BIM consiste em um conjunto estruturado de políticas, processos e ferramentas que formam uma metodologia para gerenciar os dados digitais e os projetos de construção ao longo do ciclo de vida do edifício.

Esse recurso surgiu "após a inovação do CAD 3D paramétrico", tecnologia que foi amplamente adotada em setores como aeroespacial, automotivo e de manufatura ([MONTEIRO, 2011](#)). O BIM vai além de representações tridimensionais que incluem dimensões, volumes e áreas, incorporando informações adicionais, como acabamento, custos, fabricantes e relações com outros elementos. Isso permite extrair informações de maneira mais eficiente e, em caso de alterações estruturais, atualizar todos os dados do projeto de forma centralizada. Apesar de seu caráter inovador, a implementação do BIM demanda equipes bem preparadas, tanto na etapa de controle quanto na de acompanhamento, o que pode elevar o custo inicial, embora proporcione uma gestão mais eficiente durante todo o ciclo de vida da construção ([COELHO; NOVAES, 2018](#)).

Entre os principais benefícios do BIM está a possibilidade de trabalhar com várias dimensões em um único projeto. Com base nas informações adicionadas ao modelo, é possível reduzir falhas, otimizar prazos e garantir maior controle sobre as atualizações ([PARREIRA, 2013](#)).

Segundo ([EASTMAN; TEICHOLZ; SACKS, 2011](#)), o BIM possui sete dimensões que podem ser exploradas independentemente, desde que o modelo 3D já esteja desenvolvido. Isso significa, por exemplo, que uma análise 6D pode ser realizada sem a necessidade de uma inspeção 4D, como ilustrado na Figura 1.

- **BIM 3D - Modelo Virtual 3D:** Ampliando o conceito de modelagem 2D, o BIM 3D permite a visualização completa dos elementos em três dimensões (X, Y e Z), além de identificar colisões utilizando algoritmos originados da computação gráfica. Essa funcionalidade reduz significativamente custos na fase de design e na execução da obra.
- **BIM 4D - Programação:** Esta dimensão organiza o modelo 3D em etapas, atribuindo sequências de montagem aos elementos. Além de possibilitar a visualização das fases do projeto, ela permite simular cronogramas e planejar prazos de entrega com maior precisão.
- **BIM 5D - Estimativa de Custos:** Proporciona uma rápida estimativa de custos para diferentes fases do projeto, gerando relatórios financeiros e permitindo comparar alternativas em termos de tempo de execução e investimentos.
- **BIM 6D - Sustentabilidade:** Essa dimensão possibilita integrar informações relacionadas à proteção ambiental e ao consumo de energia, otimizando os custos e investimentos durante e após a construção.
- **BIM 7D - Gerenciamento de Instalações:** Um banco de dados detalhado é gerado para cada elemento da construção, incluindo estruturas, acabamentos e equipamentos. Assim, é possível programar manutenções, verificar garantias e gerenciar componentes ao longo da vida útil da edificação.

Devido às mudanças significativas que o BIM exige em comparação ao CAD, a resistência à sua implementação é compreensível. Conforme Coelho, o uso de ferramentas tradicionais, como o AutoCAD®, representa um obstáculo, pois a transição para novos *softwares* implica na reconfiguração de rotinas e no aprendizado de novas ferramentas (COELHO; NOVAES, 2018). Souza destaca ainda que a metodologia BIM exige *softwares* mais avançados, que geram volumes elevados de dados, demandando hardwares de alto desempenho, o que dificulta a adoção em muitas empresas (SOUZA, 2009).

Segundo Eastman, o conceito BIM não está vinculado a um único *software*, já que atender a todo o ciclo de vida de uma edificação em uma única aplicação seria inviável devido à complexidade e rigidez do processo (EASTMAN; TEICHOLZ; SACKS, 2011). Em vez disso, a criação de modelos BIM depende de diversas ferramentas especializadas, que frequentemente apresentam problemas de compatibilidade, dificultando a troca de dados (IBRAHIM ROBERT KRAWCZYK, 2004). Bottega relata que, mesmo após a adoção de plataformas BIM, a transferência de arquivos é prejudicada por questões como o tamanho dos dados e a diversidade de *softwares* utilizados (BOTTEGA, 2012).



Figura 1 – Representação do BIM - Fonte: [HOCH](#)

2.2 Aprendizado de Máquina

O Aprendizado de Máquina (AM) é um ramo da Inteligência Artificial (IA) que permite aos computadores aprenderem comportamentos, identificarem padrões e tomarem decisões com interação mínima de seres humanos ([Mitchell, 1997](#)). A partir de um conjunto de dados, os algoritmos passam por um processo de treinamento, gerando um modelo que possa lidar com novas situações e resolver problemas ([TAN; STEINBACH; KUMAR, 2006](#)).

Existem três principais tipos de aprendizagem:

- **Aprendizagem supervisionada:** ocorre quando um conjunto de dados previamente rotulados é utilizado para treinar o modelo, permitindo que ele preveja rótulos desconhecidos. Esse tipo de aprendizagem é capaz de tomar decisões precisas quando novos dados rotulados são fornecidos. Para estimar rótulos, isso pode ser feito por meio de dois tipos principais de tarefa: classificação e regressão. A classificação é utilizada para categorizar os dados em classes específicas, enquanto a regressão busca prever valores contínuos baseados em dados históricos ([Faceli et al., 2011](#)).
- **Aprendizagem não supervisionada:** ocorre quando um conjunto de dados não possui nenhum rótulo associado. Nesse contexto, o objetivo é identificar padrões ou similaridades entre os dados analisados. Essa forma de aprendizagem é amplamente aplicada em problemas de agrupamento, recomendações e detecção de anomalias. Nesse caso, mesmo que os dados não sejam rotulados, eles podem ser classificados como similares ou agrupados com base nos padrões identificados ([Faceli et al., 2011](#)).
- **Aprendizagem por reforço:** consiste em um processo de tentativa e erro, onde a máquina aprende quais ações resultam em maiores recompensas. A cada decisão tomada, o agente

recebe uma recompensa ou uma punição, e o objetivo é maximizar as recompensas ao longo do tempo. Essa abordagem é amplamente utilizada em jogos e aplicações onde a máquina precisa tomar decisões sequenciais, como, por exemplo, em sistemas de controle ou simulações em ambientes complexos (Cerri; Carvalho, 2017).

Nos problemas de classificação, o algoritmo busca criar um modelo capaz de generalizar os padrões extraídos do conjunto de treinamento para categorizar novas instâncias de dados não vistos. Já nos problemas de regressão, o objetivo do algoritmo é estimar valores contínuos, como, por exemplo, prever as vendas futuras de um produto ou estimar o preço de um imóvel com base em características históricas (Guimarães; Meireles; Almeida, 2019).

Um exemplo de problema de classificação é o mapeamento de uma imagem para classificar um indivíduo como homem ou mulher, ou a utilização de um classificador de sentimentos, onde o modelo pode determinar se um texto é positivo ou negativo. Por outro lado, exemplos de problemas de regressão incluem algoritmos que preveem vendas de produtos, estimam o valor de um imóvel ou calculam a expectativa de vida de um país.

Esse processo de aprendizagem é estruturado em etapas conhecidas como Fluxo de Aprendizagem de Máquina, que incluem: obtenção dos dados; preparação, limpeza e manipulação dos dados; escolha e treinamento do modelo; teste dos dados; avaliação do modelo; e melhorias do modelo (TAN; STEINBACH; KUMAR, 2006). A qualidade dos dados desempenha um papel fundamental no AM, pois o conhecimento extraído é diretamente influenciado pela qualidade das informações fornecidas (Guimarães; Meireles; Almeida, 2019). Portanto, é necessário um processo rigoroso de pré-processamento dos dados antes da aplicação dos algoritmos de AM.

O pré-processamento de dados é essencial, especialmente devido à presença de ruídos nos dados. Essa etapa pode incluir a detecção e remoção de anomalias, o tratamento de valores ausentes ou até mesmo a exclusão de atributos irrelevantes. Todo esse esforço visa aumentar a qualidade do treinamento do modelo. O sucesso do pré-processamento depende diretamente da habilidade do profissional em identificar problemas nos dados e utilizar métodos adequados para solucioná-los (Neves, 2003).

Além das dificuldades no pré-processamento, também podem surgir problemas nas fases de treinamento e teste. Um dos desafios mais comuns é o overfitting, que ocorre quando o modelo se ajusta excessivamente aos dados de treinamento, tornando-se incapaz de generalizar para novos dados. Outro problema é o underfitting, quando o modelo não consegue capturar padrões suficientes nos dados, resultando em um desempenho insatisfatório (RUSSELL; NORVIG, 2010).

Uma vez que o modelo é construído, torna-se essencial medir seu desempenho. Entre as métricas mais comuns utilizadas para esse propósito estão:

- **A acurácia (ACC ou *accuracy*):** É uma métrica muito utilizada na literatura, e sua fórmula calcula o número de acertos dividido pelo número total (número de acertos +

número de erros). Essas informações são retiradas da matriz de confusão (Equação 2.1). A matriz de confusão (Figura 2) é uma tabela que mostra o desempenho de um algoritmo de classificação. Se o problema de classificação for binário, ela apresentará quatro informações, que são:

$$ACC = \frac{(TP + TN)}{(TP + FN + FP + TN)} \quad (2.1)$$

- **Verdadeiro Positivo (TP ou *True Positive*)**: Ocorre quando a classe que estamos buscando foi prevista corretamente. Exemplo: a pessoa está doente, e o modelo previu corretamente que está doente;
- **Falso Positivo (FP ou *False Positive*)**: Ocorre quando a classe que estamos buscando foi prevista incorretamente. Exemplo: a pessoa não está doente, mas o modelo previu incorretamente que está doente;
- **Verdadeiro Negativo (TN ou *True Negative*)**: Ocorre quando a classe que não estamos buscando prever foi prevista corretamente. Exemplo: a pessoa não está doente, e o modelo previu corretamente que não está doente;
- **Falso Negativo (FN ou *False Negative*)**: Ocorre quando a classe que estamos buscando foi prevista incorretamente. Exemplo: a pessoa está doente, mas o modelo previu incorretamente que não está doente.

Figura 2 – Matriz de confusão.

		VALOR VERDADEIRO	
		positivo	negativo
VALORES PREVISTOS	positivo	TP	FN
	negativo	FP	TN

Fonte: Própria (2024).

- A **precisão (PRE ou *precision*)**: É utilizada para trabalhar somente com os valores positivos, e sua fórmula é calculada pelos valores verdadeiros positivos divididos pelo total de predições positivas (verdadeiro positivo + falso positivo) (Equação 2.2).

$$PRE = \frac{TP}{(TP + FP)} \quad (2.2)$$

- **A revocação (REC ou *recall*):** É utilizada para encontrar exemplos de uma classe, e sua fórmula é calculada pelos valores verdadeiros positivos divididos pelo número de resultados que deveriam ser apresentados (verdadeiro positivo + falso negativo) (Equação 2.3).

$$REC = \frac{TP}{(TP + FN)} \quad (2.3)$$

- **A medida F1 (F1S ou *F1-Score*):** É a junção da precisão com a revocação, resultando em um número único que indica a qualidade geral do modelo (Equação 2.4). O cálculo da medida F1 é dado por:

$$F1S = \frac{2 \cdot PRE \cdot REC}{PRE + REC} \quad (2.4)$$

Outro conceito relevante no contexto de AM é a transferência de aprendizado (Transfer Learning), que envolve a reutilização de modelos treinados para resolver problemas relacionados em outros domínios. Essa abordagem busca aproveitar o conhecimento adquirido em uma tarefa para facilitar o aprendizado em outra tarefa, mesmo que os conjuntos de dados sejam diferentes (Shao; Zhu; Li, 2015).

Finalmente, o Aprendizado Profundo (*Deep Learning*) é uma técnica avançada de AM que utiliza redes neurais profundas, inspiradas nos neurônios do cérebro humano. Essas redes são compostas por camadas de entrada, ocultas e saída, e transformam dados em informações úteis por meio de processos sequenciais. O aprendizado profundo é amplamente utilizado em aplicações como reconhecimento de imagens, processamento de linguagem natural e sistemas de recomendação (Arroyo-Figueroa; Sucar; Villavicencio, 2000)

2.3 PLN

Na década de 1950, Alan Turing (conhecido como o pai da computação) propôs o Teste de Turing, um experimento no qual um sistema tenta determinar se está interagindo com um ser humano ou uma máquina. Esse teste marcou o início das reflexões sobre o Processamento de Linguagem Natural (PLN) (RUSSELL; NORVIG, 2010).

O Processamento de Linguagem Natural (PLN) tem como objetivo permitir que computadores realizem tarefas relacionadas à linguagem humana, criando uma interação entre máquinas e humanos por meio de linguagem falada e/ou escrita (Jurafsky; Martin, 2009). Segundo Liddy, o PLN é uma técnica que analisa e representa textos de maneira natural, buscando alcançar um processamento de linguagem semelhante ao do ser humano em diversas aplicações (Liddy, 1998).

Entretanto, as linguagens naturais, como o Português e o Inglês, não possuem regras determinísticas e apresentam ambiguidades. Isso as diferencia das linguagens formais, como

Python ou Java, que possuem conjuntos de regras bem definidos. Nas linguagens formais, frases precisam seguir a gramática para serem válidas, e sua semântica é projetada para evitar ambiguidades.

Os sistemas de PLN podem ser classificados de acordo com o nível de unidade linguística processada ([Liddy, 1998](#)), como segue:

- **Nível Fonológico:** Interpretação dos sons da língua. Um exemplo é a diferenciação entre as palavras "faca" e "vaca" por meio do som.
- **Nível Morfológico:** Análise da função, estrutura e flexão das palavras. Por exemplo, identificar que "andar" é um verbo que indica ação.
- **Nível Lexical:** Análise do significado das palavras, incluindo suas relações semânticas. Por exemplo, compreender que "pedra" está associada a palavras como "pedregulho", "pedraria" e "pedrinha".
- **Nível Sintático:** Estudo da estrutura das frases e de como são organizadas para transmitir significado.
- **Nível Semântico:** Análise do significado das palavras e frases, como identificar que o verbo "levar" pode significar transportar, carregar ou guiar, dependendo do contexto.
- **Nível de Discurso:** Interpretação de textos mais extensos, considerando sua estrutura e significado.
- **Nível Pragmático:** Compreensão do contexto comunicacional. Por exemplo, quando alguém diz "Nossa, as janelas estão todas abertas! Como está frio aqui!" e outra pessoa responde "Vou fechá-las", é possível inferir a intenção implícita da primeira pessoa.

De acordo com Bulengo, o PLN pode ser dividido em quatro etapas principais: análise morfológica, análise sintática, análise semântica e análise pragmática, que são executadas nessa ordem ([Bulegon; Moro, 2010](#)). Santos, no entanto, afirma que geralmente são utilizadas apenas as três primeiras análises. Na análise morfológica, por exemplo, identificam-se substantivos, verbos e adjetivos, gerando um "dicionário" que é usado na análise sintática para compreender as relações entre palavras, como sujeito, predicado e complementos. Por fim, a análise semântica examina os termos ambíguos e seus significados no contexto ([Santos et al., 2015](#)).

Na prática, raramente todos os níveis são implementados em um sistema, devido à complexidade crescente à medida que se avança nos níveis. No entanto, o PLN tem ampla aplicação, como em reconhecedores e sintetizadores de fala, corretores ortográficos, tradutores automáticos, sistemas de geração e resumo de texto, extração de informações e interfaces de linguagem natural ([Vieira; Lima, 2001](#)).

Com o crescimento do PLN, pesquisadores têm buscado fundamentação em diversas áreas, como Filosofia da Linguagem, Psicologia, Matemática, Linguística e Ciência da Computação. Assim como no Aprendizado de Máquina (AM), o PLN exige etapas de pré-processamento que estruturam a linguagem, reduzindo o vocabulário e melhorando o desempenho dos modelos (Rezende; Marcacini; Moura, 2011).

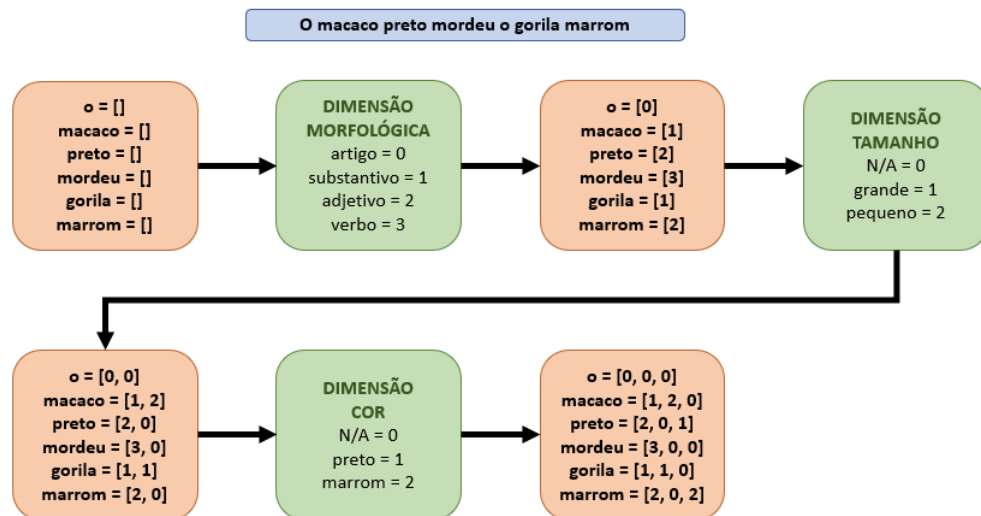
Entre as principais tarefas de pré-processamento no PLN, destacam-se:

- **Normalização:** Remoção de caracteres especiais, tags HTML/CSS, e transformação de letras maiúsculas em minúsculas.
- **Tokenização:** Separação do texto em unidades mínimas chamadas "tokens", que podem ser palavras, símbolos ou caracteres.
- **Stemização:** Redução das palavras ao seu radical, como transformar "meninas" em "menin".
- **Lematização:** Redução das palavras à sua forma base ou lema, como transformar "meninas" e "meninos" em "menino".
- **Remoção de Stopwords:** Exclusão de palavras frequentes e pouco informativas, como "a", "de", "e".
- **Correção Ortográfica:** Tratamento de erros de digitação e abreviações para evitar problemas nos modelos.

A representação por palavras é conhecida como vetores de palavras (ou *word embeddings*), onde a ideia é transformar as palavras em números, mantendo suas relações com outras palavras. Esse processo se baseia em um *corpus* (um conjunto de textos a partir do qual as relações serão aprendidas). A primeira etapa é dividir o texto em palavras; em seguida, cada uma delas é representada por vetores, onde o tamanho do vetor corresponde ao número de dimensões definidas pelo *corpus*. A Figura 3 ilustra o *word embeddings* com 3 dimensões (morfológica, tamanho e cor), representando um conceito completo.

As vantagens do *word embeddings* incluem:

- **Representação Sem Contexto:** As palavras são analisadas independentemente do contexto imediato, o que pode limitar certas nuances, mas ainda permite categorizações úteis.
- **Relações Semânticas:** Em um espaço n-dimensional, palavras com significados similares estão próximas umas das outras, como "macaco" e "gorila".
- **Operações Matemáticas:** É possível realizar operações vetoriais para descobrir relações semânticas entre palavras. Por exemplo, somar um vetor associado a uma característica específica a outro vetor pode resultar em uma nova palavra relacionada.

Figura 3 – Exemplo da transformação de palavras em *word embeddings*.

Fonte: Própria (2021).

Enquanto no exemplo acima a intuição foi usada para criar as dimensões, na Inteligência Artificial modelos baseados em Aprendizado Profundo são comumente aplicados. Eles classificam palavras em dimensões com base no contexto em que aparecem. Quanto maior o *corpus*, mais precisas e eficientes tendem a ser essas representações (Bengio et al., 2003).

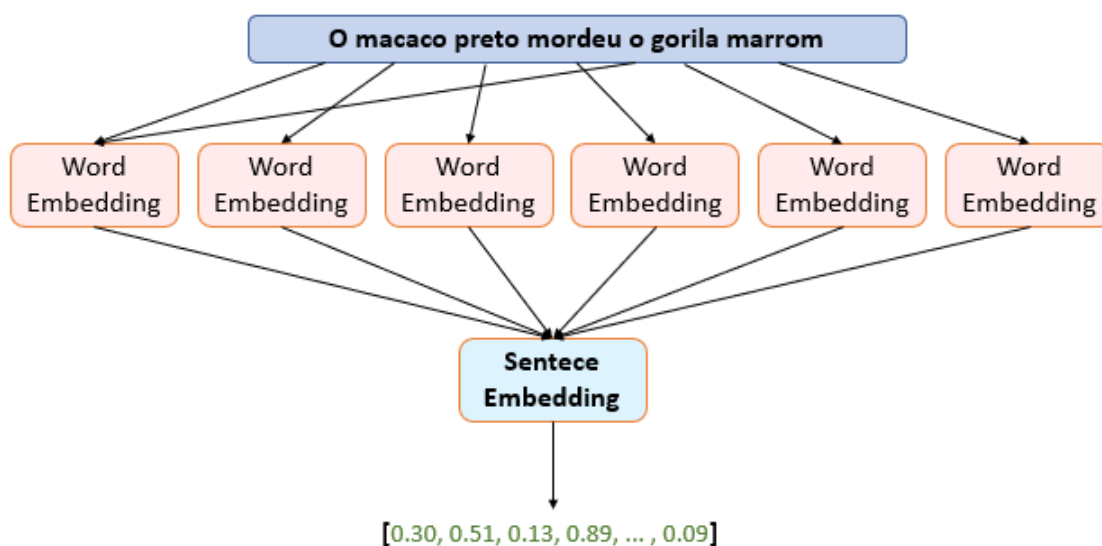
Além do conceito de *word embeddings*, existe o *sentence embeddings*, que considera não apenas as palavras, mas também a ordem em que aparecem na frase (Conneau et al., 2017). No *sentence embeddings*, cada palavra é representada por seu *word embedding*, e o conjunto dessas palavras forma uma frase completa, com informações semânticas representadas por vetores, facilitando a compreensão do contexto e da intenção da frase (Figura 4).

Diversas técnicas foram desenvolvidas para representar palavras vetorialmente (*word embeddings*), diferenciando-se por suas arquiteturas e bases de dados usadas no treinamento (Martin; Jurafsky, 2009). Além disso, essas técnicas podem ser transferidas para domínios semelhantes em um processo chamado Transferência de Aprendizado, que será discutido em outra seção.

No contexto do PLN moderno, representações vetoriais, como *word embeddings* e *sentence embeddings*, têm sido amplamente utilizadas. Modelos como Word2Vec, GloVe, fastText e OpenAIEmbeddings destacam-se no campo (Devlin et al., 2019). O OpenAIEmbeddings, em particular, apresenta um desempenho robusto em diversas tarefas de PLN.

2.4 Métricas de Similaridade Textual

A avaliação de saídas geradas por LLMs em tarefas de conversão de texto requer a comparação entre o texto produzido pelo modelo e um texto de referência. Como nem sempre a

Figura 4 – Exemplo da transformação de *word embeddings* para *sentence embeddings*.

Fonte: Própria (2021).

correspondência exata é desejável — muitas vezes paráfrases corretas devem ser aceitas —, são utilizadas métricas de similaridade que se dividem, de forma geral, em três famílias: *lexicais*, *semânticas* e *de distância* (Jurafsky; Martin, 2009).

As métricas **lexicais** comparam as cadeias de caracteres ou os tokens presentes nos textos, sem considerar o significado das palavras. Uma das mais difundidas é a baseada na distância de Levenshtein, que conta o número mínimo de operações de edição (inserção, remoção ou substituição) necessárias para transformar uma cadeia em outra. A biblioteca *FuzzyWuzzy* oferece variantes dessa medida, como o *partial ratio*, adequado para casos em que uma string é fragmento da outra. Outra métrica lexical clássica é a similaridade do cosseno aplicada a vetores TF-IDF (*Term Frequency–Inverse Document Frequency*), que pondera os termos pela sua importância relativa no corpus (Manning; Raghavan; Schütze, 2008).

As métricas **semânticas** apoiam-se em *sentence embeddings*: representam cada texto como um vetor em um espaço de alta dimensionalidade, treinado para aproximar sentenças com significado parecido. A similaridade é então calculada pelo cosseno entre os vetores. Modelos como o *Sentence-BERT* (SBERT) (REIMERS; GUREVYCH, 2019) estendem o BERT para gerar tais representações de forma eficiente. Para o português, destaca-se o BERTimbau (Souza; Nogueira; Lotufo, 2020), especializado em textos em português brasileiro, com variantes ajustadas para o domínio jurídico. Modelos multilíngues, como o *paraphrase-multilingual-mpnet-base*, permitem comparações entre línguas diferentes em um mesmo espaço vetorial.

As métricas **de distância** medem o custo de transformar um texto em outro no espaço de *embeddings*. O *Word Mover's Distance* (WMD) (KUSNER et al., 2015) formaliza esse problema como um transporte ótimo entre as palavras de duas sentenças, ponderado pela

distância euclidiana entre seus *embeddings*. Para o português, o WMD pode ser calculado com *embeddings* do FastText (Bojanowski et al., 2017) ou com modelos do projeto NILC, treinados em corpora em português. Valores normalizados de WMD aproximam-se de 1 quando as sentenças são semanticamente próximas e tendem a 0 quando distantes.

Essas três famílias são complementares: as lexicais penalizam reformulações que preservam o significado, enquanto as semânticas e as de distância capturam equivalências apesar de variações de superfície. A combinação das três fornece um retrato mais completo da qualidade das saídas de LLMs e é amplamente utilizada na literatura de PLN aplicada (REIMERS; GUREVYCH, 2019).

2.5 Redes Neurais Artificiais

O sistema nervoso humano é composto por neurônios, que desempenham a função de transmitir informações essenciais para o funcionamento, comportamento e raciocínio do corpo humano (Tafner; Xerez; Filho, 1995). As Redes Neurais Artificiais (RNAs) são técnicas computacionais inspiradas na estrutura neural do ser humano, adquirindo conhecimento por meio da experiência (Haykin, 1999). Assim, uma rede neural artificial pode ser compreendida como uma abstração da rede neural biológica, com o objetivo de modelar processos de aprendizado e resolver problemas complexos.

O primeiro modelo que simulou uma rede neural biológica foi desenvolvido em 1943 pelos psiquiatras Warren McCulloch e Walter Pitts. Esse trabalho apresentou detalhes sobre o comportamento e a memória dos neurônios biológicos em uma forma artificial, mas não abordou como o aprendizado poderia ser implementado (McCulloch; Pitts, 1943)). Já em 1949, Donald Hebb conduziu o primeiro estudo voltado à fase de aprendizado das redes neurais artificiais, demonstrando que os pesos em cada conexão entre neurônios resultavam em uma força de excitação, similar à encontrada nos neurônios biológicos (Hebb, 1949). Entretanto, esse modelo ainda não possuía a capacidade de reconhecer padrões.

Em 1958, Rosenblatt desenvolveu o primeiro modelo capaz de detectar padrões e características, demonstrando que, ao modificar os pesos conectados aos neurônios, o modelo poderia ser treinado para classificar padrões (Rosenblatt, 1958). Este modelo ficou conhecido como perceptron e apresenta uma arquitetura composta por três camadas:

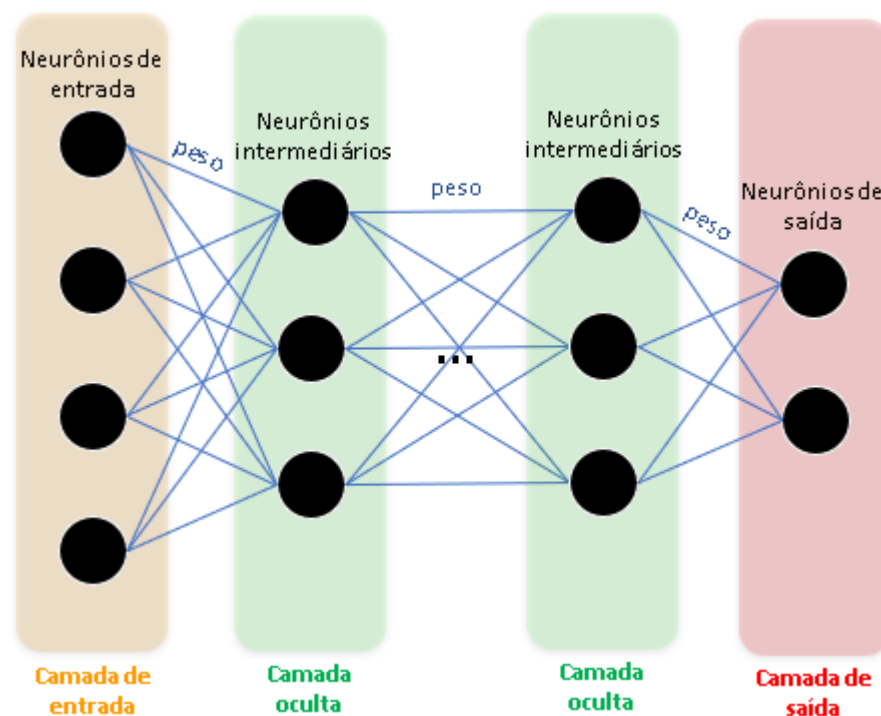
- **Camada de entrada:** Responsável por receber informações do ambiente externo e repassá-las às camadas intermediárias.
- **Camadas intermediárias (ou ocultas):** Processam os sinais recebidos, ajustam os pesos e enviam as informações para a camada seguinte.
- **Camada de saída:** Produz e apresenta o resultado final.

Contudo, quando utilizado para classificar padrões, uma rede perceptron de três camadas é limitada a encontrar uma única solução. Para resolver problemas que exigem múltiplas soluções, Minsky e Papert propuseram a inclusão de múltiplas camadas intermediárias, resultando no modelo conhecido como perceptron multicamadas (*Multi-layer Perceptron – MLP*) (Minsky; Papert, 1969).

O MLP é composto por um conjunto de neurônios conectados em paralelo, com cada conexão associada a um peso que reflete sua influência na rede. A partir da interação em camadas intermediárias, onde a saída de um neurônio se torna a entrada de outro, obtém-se um comportamento inteligente, capaz de reconhecer padrões e características (Soares; Teive, 2015).

A estrutura de um neurônio em uma rede MLP processa informações recebidas do ambiente externo (valores x) associadas a pesos (w). Uma soma ponderada dos produtos de x e w é então calculada, somando-se o limiar de ativação (bias ou polarização). O valor resultante, chamado de ponto de ativação (u), é avaliado por uma função de ativação que limita a amplitude da saída do neurônio, geralmente dentro do intervalo $[0, 1]$, podendo ser interpretado como uma probabilidade (Goodfellow; Bengio; Courville, 2016).

Figura 5 – Rede Neural MLP.

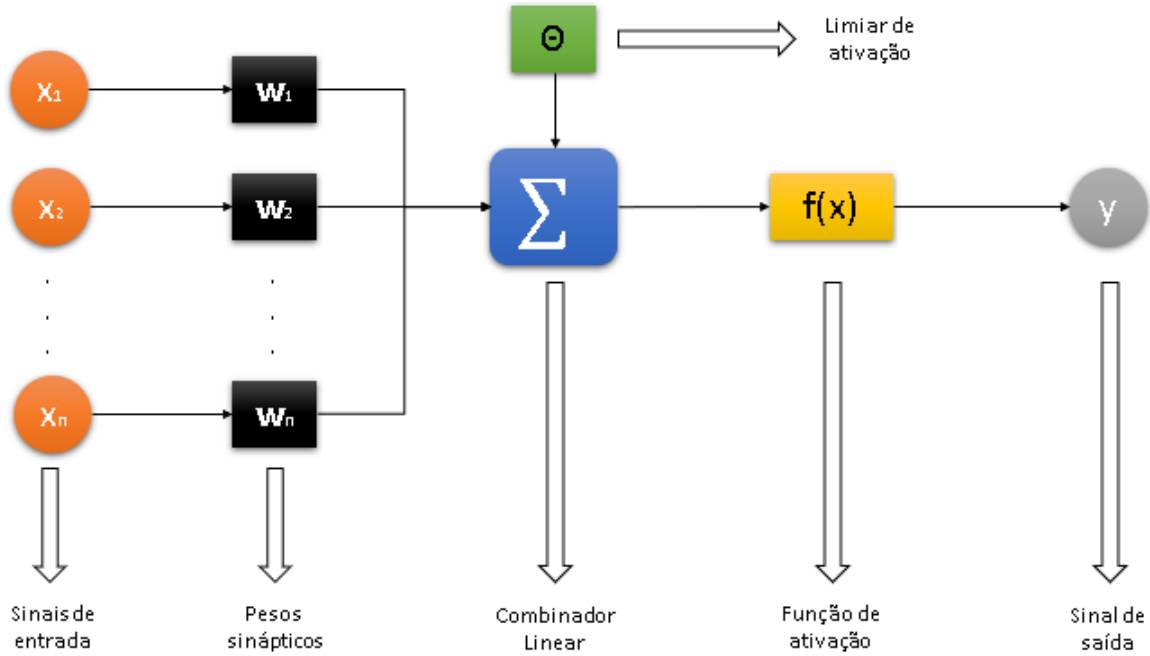


Fonte: Própria (2024).

A Figura 6 demonstra o processamento de informações por um neurônio em uma rede MLP, onde, dado um conjunto de valores ' x ' (que representam as informações do mundo externo), esses valores são associados a pesos ' w '. A partir disso, é efetuada uma soma ponderada dos produtos ' x ' e ' w ', à qual é somado o limiar de ativação (bias ou polarização), como mostrado na

Equação 2.5.

Figura 6 – Estrutura de um neurônio no MLP.

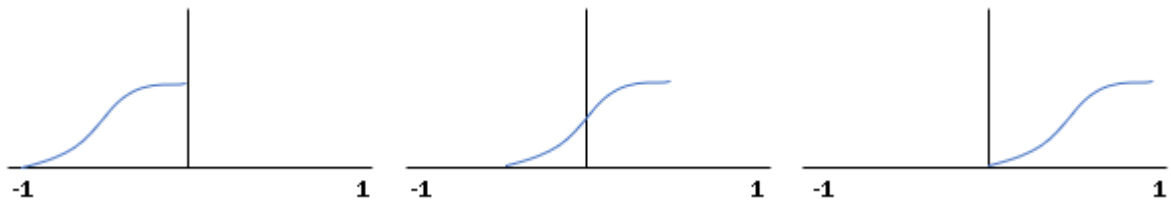


Fonte: Própria (2024).

$$u = \sum_{i=1}^n x_i \cdot w_i + \theta \quad (2.5)$$

O bias (θ) é responsável pelo deslocamento inicial da função de ativação no plano onde ela é representada. Caso o bias seja positivo, o movimento é realizado para a esquerda, diminuindo o valor do eixo x. Caso seja negativo, o movimento é realizado para a direita, aumentando o valor do eixo x (Rojas, 1996), como ilustrado na Figura 7.

Figura 7 – Exemplo do deslocamento do gráfico através da mudança da constante bias.



Fonte: Própria (2024).

O valor u obtido, chamado de ponto de ativação, é avaliado pela função de ativação, que é responsável por calcular o sinal da saída do neurônio. A função de ativação, também conhecida como função de transferência, tem o objetivo de limitar a amplitude da saída do neurônio. Ou seja,

o valor obtido estará no intervalo entre 0 e 1, podendo ser interpretado como uma probabilidade do resultado (Goodfellow; Bengio; Courville, 2016).

Existem diversos tipos de funções de ativação, e não há uma regra fixa para determinar qual é melhor ou pior. A escolha depende do problema, embora, dependendo das propriedades específicas, seja possível optar por uma função que facilite e acelere a convergência da rede neural (Goodfellow; Bengio; Courville, 2016).

As funções de ativação mais populares são:

- **Função de Etapa Binária** ($f(x) = 1$, se $x \geq 0$ e $f(x) = 0$, se $x < 0$): Essa função determina se o neurônio deve ou não estar ativo. Caso o valor ' u ' seja maior que um limite pré-determinado, o neurônio é ativado; caso contrário, ele permanece desativado. Embora útil em classificadores binários, é mais utilizada como referência teórica, pois, no processo de retropropagação (backpropagation), os gradientes das funções de ativação retornam zero, o que impede a melhoria dos resultados (Feng; Lu, 2019).
- **Função Linear** ($f(x) = ax$): Nessa função, cada camada passa por uma transformação linear. No entanto, essa função possui limitações semelhantes à função de etapa binária, já que sua derivada é constante, independentemente do valor da entrada. Isso impede melhorias durante o backpropagation (Feng; Lu, 2019).
- **Função Sigmoid** ($f(x) = 1/(1 + e^{-x})$): Essa função comprime os valores de saída para o intervalo entre 0 e 1, o que é útil para classificar uma classe específica. Embora não seja linear e permita saídas não lineares em redes neurais, apresenta dois problemas principais: 1) os gradientes podem se aproximar de zero, o que prejudica o aprendizado da rede, e 2) os valores de saída são sempre positivos, o que pode não ser desejável em todos os casos. Apesar disso, ela é amplamente utilizada em classificadores (Feng; Lu, 2019).
- **Função Tanh** ($f(x) = 2/(1 + e^{(-2x)}) - 1$): É uma variação escalonada da função Sigmoid, mas simétrica em relação à origem, variando de -1 a 1. Isso elimina o problema dos valores positivos constantes. As demais propriedades são semelhantes às da Sigmoid (Feng; Lu, 2019).
- **Função ReLU** ($f(x) = \max(0, x)$): É a função de ativação mais amplamente utilizada no design de redes neurais. Por ser não linear, facilita a propagação dos erros e melhorias durante o treinamento. Sua principal vantagem é não ativar todos os neurônios simultaneamente, tornando a rede mais eficiente. No entanto, pode sofrer com o problema de gradientes que convergem para zero (Feng; Lu, 2019).
- **Função Leaky ReLU** ($f(x) = ax$, se $x < 0$ e $f(x) = x$, se $x \geq 0$): Uma variação da ReLU que corrige o problema do gradiente zero. Quando ' x ' < 0, é aplicada uma componente linear em vez de zero, garantindo um gradiente constante mesmo para valores negativos. É

especialmente útil em camadas ocultas e para lidar com neurônios defeituosos (Feng; Lu, 2019).

- **Função Softmax:** É uma extensão da Sigmoid, projetada para problemas de classificação com múltiplas classes. Transforma as saídas de cada classe em valores entre 0 e 1, normalizando-as para que sua soma seja igual a 1. Assim, é possível interpretar os resultados como probabilidades de pertencimento a cada classe (Sensoy; Kaplan; Kandemir, 2018).

O treinamento de uma rede neural artificial multicamadas consiste em ajustar os pesos sinápticos e o bias de modo que as saídas se aproximem das saídas esperadas. Esse processo ocorre em duas etapas (RUSSELL; NORVIG, 2010):

- **Propagação (forward-propagation):** Nesta etapa, aplica-se a Equação 3.7 em cada neurônio. O fluxo de dados começa na camada de entrada, atravessa as camadas ocultas e finaliza na camada de saída (Hafemann, 2014).
- **Retropropagação (backpropagation):** Após a etapa de propagação, os valores de saída são comparados com os valores desejados usando uma função custo, que mede o desempenho da rede. Caso o desempenho seja insatisfatório, inicia-se a retropropagação para minimizar o erro, ajustando os pesos e os bias. Isso é feito utilizando o método do Gradiente Descendente (Hafemann, 2014).
 - **Função Custo (Loss):** Mede o quão distante está a saída da rede em relação à saída esperada. Para regressão, a função mais comum é o Erro Quadrático Médio (MSE). Para classificação, é a Entropia Cruzada (McCaffrey, 2013).
 - **Erro Quadrático Médio (MSE):** Mede a diferença entre os valores previstos e os reais, elevando ao quadrado e calculando a média (Equação 2.6) (Willmott, 1981).

$$MSE = \frac{1}{n} \sum_{i=1}^n (\theta_{ip} - \theta_i)^2 \quad (2.6)$$

- **Raiz do Erro Quadrático Médio (RMSE):** Similar ao MSE, mas calcula a raiz quadrada da média dos erros (Equação 2.7) (Chai; Draxler, 2014).

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n e_i^2} \quad (2.7)$$

- **Erro Médio Absoluto (MAE):** Mede o desvio médio entre valores previstos e reais, com pesos iguais para todos os desvios (Equação 2.8) (Willmott, 1981).

$$MAE = \frac{\sum_{i=1}^n |O_i - P_i|}{n} \quad (2.8)$$

- **Entropia Cruzada:** Mede a distância entre os vetores de saída e os valores esperados (Equação 2.9) (Nasr; Badr; Joun, 2002).

$$E(w) = \frac{1}{n} \cdot \sum_{i=1}^n [t_i \cdot \log(y_i) + (1 - t_i) \cdot \log(1 - y_i)] \quad (2.9)$$

- **Gradiente Descendente:** Método de otimização que ajusta pesos e bias na direção que minimiza o erro. A cada iteração, busca-se o mínimo local da função custo (Equação 2.10) (Rumelhart; Hinton; Williams, 1986).

$$(\mathbf{w}, b) \leftarrow (\mathbf{w}, b) - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \partial_{(\mathbf{w}, b)} l^{(i)}(\mathbf{w}, b) \quad (2.10)$$

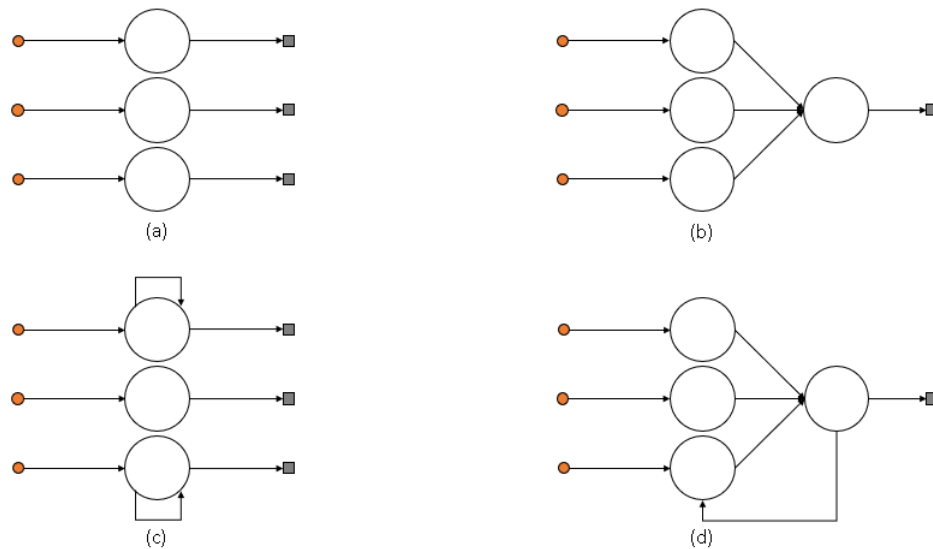
Cada ciclo de propagação e retropropagação ajusta os pesos para reduzir a função de perda, repetindo o processo até atingir o erro mínimo local (RUSSELL; NORVIG, 2010).

Um dos ramos do Aprendizado de Máquina baseado em Redes Neurais Artificiais é o Aprendizado Profundo (ou Deep Learning), que utiliza diversas camadas capazes de extrair características em diferentes níveis. Por exemplo, em um processamento de imagem, a primeira camada pode extrair formas gerais, como objetos ou rostos humanos, enquanto camadas posteriores extraem características mais específicas, como bordas e cantos. Ou seja, as camadas iniciais extraem recursos de baixo nível, enquanto as camadas mais profundas extraem recursos de alto nível e/ou médio nível (Schmidhuber, 2015).

Um dos principais benefícios da Aprendizagem Profunda é sua capacidade de extrair características automaticamente, utilizando pesos adaptáveis no kernel, o que economiza tempo ao evitar a necessidade de pré-processamento manual (Schmidhuber, 2015). Nesse contexto, uma rede neural artificial possui três tipos de camadas, e os nós (ou nodos) podem ter dois tipos de conexões, que são:

- **Redes de camada única:** ocorre quando há apenas um nó entre qualquer entrada e saída da rede (Figura 8 (a) e (c)) (Braga; Ludermir; Carvalho, 2000).
- **Redes de múltiplas camadas:** ocorre quando há mais de um neurônio entre qualquer entrada e saída da rede (Figura 8 (b) e (d)) (Braga; Ludermir; Carvalho, 2000).
- **Redes recorrentes:** ocorre quando existe pelo menos um laço de realimentação em alguma camada (Figura 8 (c) e (d)) (Braga; Ludermir; Carvalho, 2000).
- **Nó acíclico (feedforward):** ocorre quando a saída de um neurônio não é utilizada como entrada em camadas anteriores ou iguais (Figura 8 (a) e (b)) (Haykin, 2001).
- **Nó cíclico (feedback):** ocorre quando a saída de um neurônio é utilizada como entrada em camadas anteriores ou iguais (Figura 8 (c) e (d)) (Haykin, 2001).

Figura 8 – Tipos de camadas e conexões de nodos.



Fonte: Própria (2024).

Ou seja, Redes Neurais Recorrentes (Recurrent Neural Networks ou RNN) diferem substancialmente das tradicionais Redes Neurais Feedforward (FNN), pois possuem ciclos, permitindo que os dados de saída sejam alimentados de volta nas camadas anteriores. Isso torna o modelo capaz de capturar dependências temporais entre os dados (Ramakrishnan; Soni, 2018).

O ciclo presente nas RNNs permite que essas redes utilizem seu estado interno (memória) para processar entradas sequenciais, o que é essencial para tarefas que envolvem sequências temporais. Essa capacidade de armazenar informações passadas é especialmente útil em aplicações como reconhecimento de fala, processamento de linguagem natural, previsão de séries temporais, entre outras.

No entanto, devido à repetição de operações no ciclo das RNNs, que inclui uma matriz de pesos W representando os pesos de cada neurônio em uma camada, essas redes tendem a se tornar profundas. Essa profundidade pode gerar problemas como a dissipação do gradiente (gradient vanishing) e a explosão do gradiente (gradient explosion) (Goodfellow; Bengio; Courville, 2016).

A dissipação do gradiente ocorre quando os erros das últimas camadas afetam pouco os erros nas camadas iniciais, prejudicando o treinamento da rede. Isso acontece porque, durante as atualizações, os pesos e o bias das camadas iniciais não são significativamente ajustados. Por outro lado, a explosão do gradiente faz com que os gradientes nas camadas iniciais fiquem muito grandes, tornando a rede instável (Goodfellow; Bengio; Courville, 2016).

Para mitigar esses problemas, os cientistas Hochreiter e Schmidhuber propuseram o LSTM (Long Short-Term Memory ou Memória de Longo e Curto Prazo). Nesse modelo, os neurônios tradicionais foram substituídos por células de memória, permitindo que as redes RNN superem as limitações associadas à dissipação e à explosão do gradiente (Hochreiter;

Schmidhuber, 1997).

2.6 Memória Longa de Curto Prazo

O LSTM é uma evolução do RNN. O funcionamento dessa rede é estruturado de forma em cadeia, onde cada camada representa um intervalo de tempo t e a rede possui células de memória, nas quais cada célula contém portões de entrada, de esquecimento e de saída.

- **Portão de entrada (*input gate*):** Tem o objetivo de atualizar, a partir do fluxo de dados de entrada, o estado de memória da célula (Abdel-Nasser; Mahmoud, 2019).
- **Portão de esquecimento (*forget gate*):** Tem o objetivo de controlar qual informação deve permanecer na memória (Abdel-Nasser; Mahmoud, 2019).
- **Portão de saída (*output gate*):** Tem o objetivo de determinar como a saída é influenciada pelo valor de entrada e pela memória armazenada (Abdel-Nasser; Mahmoud, 2019).

Segundo (Tai; Socher; Manning, 2015), a primeira coisa que a célula do LSTM faz é decidir qual informação será removida ou mantida no estado atual da célula. Essa operação é realizada pelo portão do esquecimento, que retorna um valor entre 0 (esquecer) e 1 (manter). Todo esse processo é calculado pela função *sigmoid* (Equação 2.11).

$$f_t = \text{sigmoid}(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (2.11)$$

Em seguida, decide-se qual nova informação será armazenada. Essa decisão ocorre em duas etapas: a primeira determina qual valor deverá ser atualizado (calculada pela função *sigmoid*) (Equação 2.12), e a segunda cria um vetor de novos valores candidatos que podem ser adicionados ao estado (calculada pela função *tanh*) (Equação 2.13).

$$i_t = \text{sigmoid}(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (2.12)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (2.13)$$

Depois disso, multiplica-se o estado antigo pelos candidatos passados (C_{t-1}), para esquecer as informações desnecessárias, e multiplica-se o estado atualizado pelos candidatos atuais ($\tilde{C}_t$), para manter as informações úteis (Equação 2.14).

$$C_t = f_t \times C_{t-1} + i_t \times \tilde{C}_t \quad (2.14)$$

Por fim, a camada *sigmoid* decide quais informações da célula irão para a saída (Equação 2.15), e essas informações são multiplicadas pela função *tanh* aplicada aos candidatos atuais, permitindo que somente as informações relevantes sejam emitidas (Equação 2.16).

$$o_t = \text{sigmoid}(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (2.15)$$

$$h_t = o_t \times \tanh(C_t) \quad (2.16)$$

A LSTM possui uma estrutura computacional complexa que opera com base na ativação de diferentes funções, decidindo quais informações devem ser mantidas, alteradas ou descartadas. Essa rede tem alcançado grande sucesso na solução de tarefas sequenciais (Tai; Socher; Manning, 2015). Com esse sucesso, surgiu uma variante denominada *Bidirectional Long Short-Term Memory* (BiLSTM ou Memória Longa de Curto Prazo Bidirecional), uma rede composta por duas LSTMs funcionando em paralelo. Uma das redes percorre a sequência para frente (*Forward LSTM*), enquanto a outra percorre a sequência para trás (*Backward LSTM*), permitindo capturar informações passadas e futuras, o que aumenta o poder de aprendizado da rede (Graves; Jaitly; Mohamed, 2013).

Apesar de as redes neurais recorrentes (RNNs) apresentarem iterações eficientes, as redes *Long Short-Term Memory* (LSTM), projetadas para tarefas do tipo *sequence-to-sequence* (*seq2seq*), recebem uma sequência de dados e produzem outra. Nesse contexto, uma estrutura neural que tem ganhado destaque no mundo do Processamento de Linguagem Natural (PLN) é o *transformer*, devido ao seu desempenho em tarefas como traduções, sumarizações e legendamento de imagens, entre outras.

2.7 Transformers

Redes neurais recorrentes (Ramakrishnan; Soni, 2018), memória de curto e longo prazo (Hochreiter; Schmidhuber, 1997) e redes neurais recorrentes com portas (Chung et al., 2014), em particular, foram firmemente estabelecidas como abordagens de última geração em modelagem de sequência. Redes de modelagem de sequência (*seq2seq*) possuem uma abordagem na qual se utilizam modelos de Aprendizado de Máquina ou de Aprendizado Profundo para obter uma sequência de dados como entrada (em um domínio específico) e converte essa sequência para uma representação em outro domínio (Sutskever; Vinyals; Le, 2014).

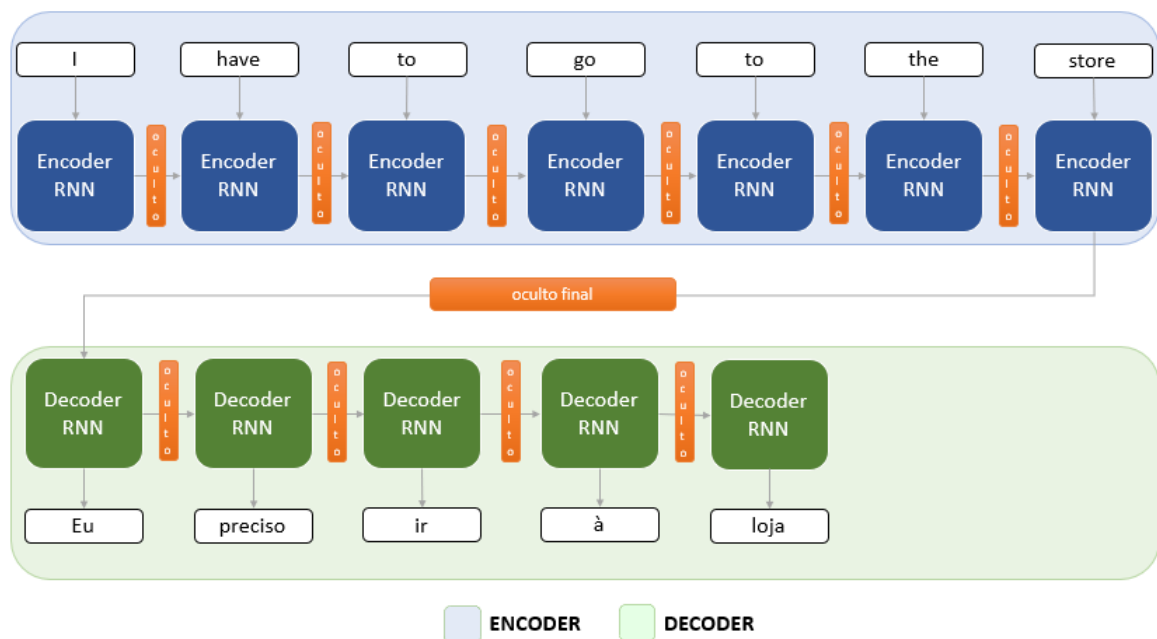
Essa abordagem normalmente é realizada por meio de dois módulos: um *encoder* (codificador) e um *decoder* (decodificador), onde o *encoder* tem o objetivo de processar a sequência de entrada e transformá-la em um contexto, enquanto o *decoder* pega o contexto e produz uma sequência de saída (Sutskever; Vinyals; Le, 2014). Um exemplo de *seq2seq* pode ser um modelo de tradução inglês-português, no qual o módulo *encoder* transforma o texto em

inglês em um "contexto livre de idioma" e, depois, o módulo *decoder* adiciona a segunda língua (português) nesse contexto.

No exemplo de um modelo de tradução inglês-português, será demonstrado o *encoder* e o *decoder* na forma de RNNs (Redes Neurais Recorrentes), onde o *encoder* processa as palavras da sequência de entrada uma a uma (de forma recorrente). A cada etapa do processamento, a palavra que entra na rede produz uma saída e um estado oculto (contexto). A saída geralmente do *encoder* é descartada, enquanto o estado oculto é passado para a próxima etapa, sendo utilizado nas palavras seguintes da sequência para produzir mais uma saída e criando um novo estado oculto (ou seja, atualizando o contexto). Esse processo se repete até a última palavra da sequência de entrada, onde o *encoder* cria o estado oculto final (contexto final).

Esse contexto final é passado para a segunda RNN, o *decoder*, que também recebe um "vetor genérico" (os valores são ajustados durante o treinamento). Esse vetor representa o início da frase de saída e, da mesma forma que o *encoder*, o *decoder* utiliza o vetor e o contexto para produzir uma saída e um estado atualizado, passando por etapas (nesse exemplo, palavras), e, em cada etapa, a saída representa a palavra traduzida até chegar na última etapa (palavra), que irá entregar a frase inteira traduzida (Figura 9).

Figura 9 – Exemplo de um modelo seq2seq para tradução do inglês para português.



Fonte: Própria (2024).

Porém, não é necessário utilizar apenas RNN para codificadores e decodificadores. É possível misturar e combinar diversos modelos, como demonstra o autor Jonghwan Mun, que utiliza um codificador CNN (*Convolutional Neural Networks*) (Simard; Steinkraus; Platt, 2003), um codificador RNN e um decodificador LSTM para um modelo que gera legendas de imagens (Mun; Cho; Han, 2016).

Como pode ser visto na Figura 9, a frase original tem 7 palavras, enquanto a frase traduzida contém 5 palavras. Por isso, dependendo da complexidade do problema, e no caso de traduções, a tarefa de traduzir não se trata de um simples alinhamento das palavras individuais. É necessário que o modelo seja capaz de focar em uma palavra, caso seja necessário, ou em um trecho específico da frase, criando assim o conceito de atenção.

Nas RNNs, o mecanismo de atenção geralmente é implementado da seguinte forma: no *encoder*, além do contexto ser representado pelo estado oculto final, ele também é representado por todos os estados ocultos produzidos após o processamento de cada palavra, fazendo com que cada estado oculto contenha o contexto acumulado até aquela palavra, o qual, após processar todas as palavras, é enviado para o *decoder* juntamente com todos os estados ocultos (Bahdanau; Cho; Bengio, 2015). Ou seja, no exemplo da Figura 9, a palavra 'go' após ser processada, possuirá o estado oculto (contexto) acumulado das palavras 'I', 'have', 'to' e 'go'.

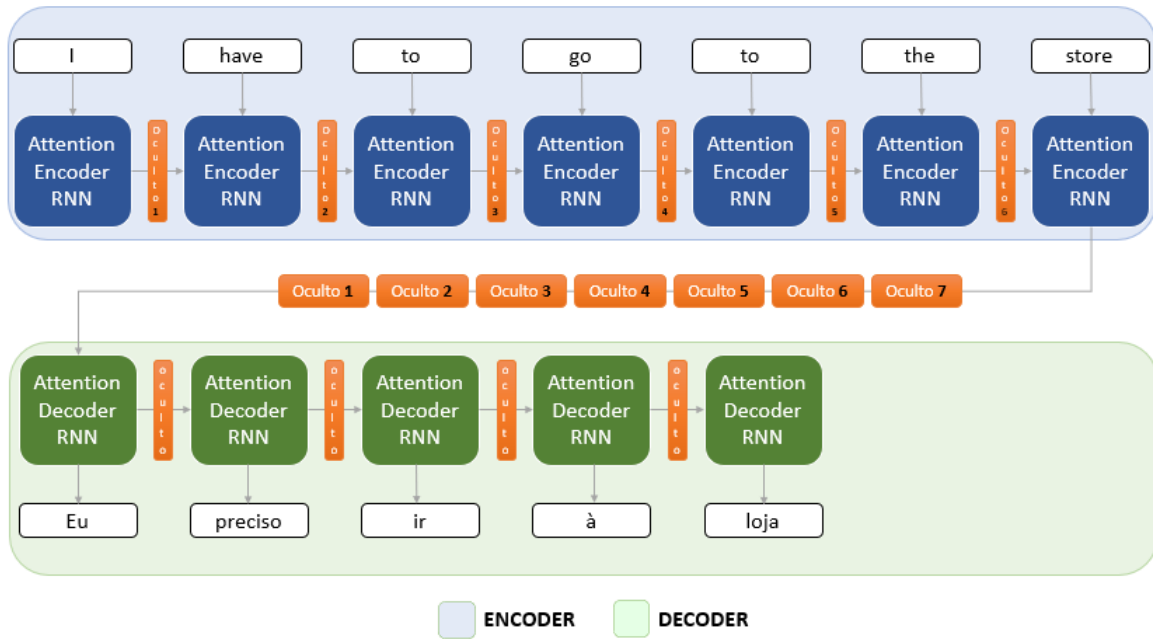
O *decoder* recebe e extrai do estado oculto final as informações necessárias para produzir o próximo estado oculto e utiliza o acúmulo de estados ocultos anteriores para calcular uma pontuação de atenção, indicando quanto foco a etapa dará para esses contextos parciais. O resultado da pontuação de atenção é multiplicado por cada estado oculto respectivo, e depois todos são somados, gerando o resultado denominado estado de contexto (Figura 10). Esse estado de contexto se espera que carregue a informação de interesse de forma amplificada, ou seja, indicando qual parte da frase o modelo deve prestar mais atenção. Por fim, o estado de contexto é concatenado com o estado final produzido nesta última etapa do *decoder*, onde o vetor é processado por uma estrutura de rede adicional do tipo totalmente conectada, produzindo então o vetor que representa a palavra no segundo idioma (português) (Bahdanau; Cho; Bengio, 2015).

De acordo com (Bahdanau; Cho; Bengio, 2015), o mecanismo de atenção obtém melhores resultados para a maioria das tarefas que necessitam "traduzir" uma sequência em outra, podendo transformar um texto em seu resumo, ou uma imagem em sua legenda. Porém, modelos recorrentes, mesmo utilizando o mecanismo de atenção (*Attention*), dada a sua natureza sequencial, acabam realizando processos computacionais intensivos para captar o contexto de uma sequência. Por isso, foi criada a arquitetura *transformer*, uma técnica que proporciona melhorias significativas tanto na eficiência computacional quanto na performance dos modelos baseados em arquiteturas recorrentes.

Assim como outras redes com mecanismo de atenção, o *transformer* também possui os módulos *encoder* e *decoder*, onde os criadores do modelo criaram 6 pilhas (*stacks*) de *encoders* e *decoders* individuais, arranjados linearmente. Cada *encoder* de cada pilha possui duas unidades (camadas) chamadas de *self-attention*, uma aplicando o mecanismo de atenção sobre o texto que ela recebe e outra que transforma o resultado da camada de *self-attention* em um *embedding* de comprimento menor (Vaswani et al., 2017).

O *decoder* é construído com as mesmas unidades do *encoder*, porém, possui uma camada adicional denominada *encoder-decoder attention*, cujo objetivo é mapear o resultado da camada

Figura 10 – Exemplo de um modelo com o mecanismo de atenção para tradução do inglês para português.

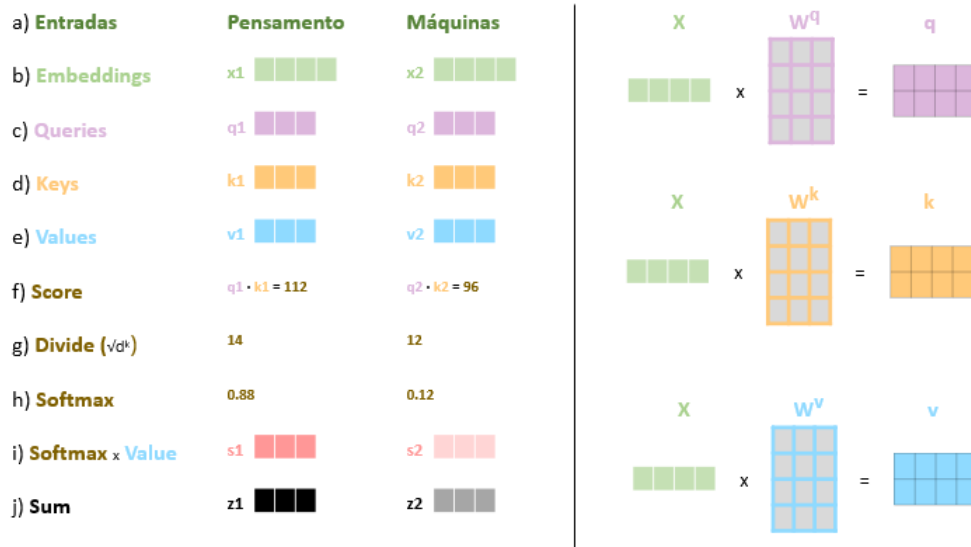


Fonte: Própria (2024).

de *self-attention* do *decoder* com os *embeddings* construídos pelo *encoder* (Vaswani et al., 2017). Com isso, enquanto nos modelos *seq2seq* as palavras são processadas sequencialmente, o *transformer* recebe o texto que deve processar tudo de uma vez, onde, através de um método independente, todas as palavras são convertidas em *embeddings*.

A camada *self-attention* (Figura 11) converte cada palavra em três vetores, denominados q (*query*), k (*key*) e v (*value*), onde é feita a multiplicação dos vetores da palavra por três matrizes (pesos) W^q , W^k e W^v , respectivamente (Figura 11 (b), (c) e (d)), cujos parâmetros são ajustados durante o treinamento da rede. Esses vetores (q , k e v) são abstrações para permitir o cálculo de atenção, que é realizado por meio de uma multiplicação entre as *queries* e as *keys*. Essa operação irá resultar em uma pontuação que será normalizada e aplicada às *values*, como mostrado na Figura 11 (e). O resultado final de uma camada de *self-attention* é uma nova sequência que leva em consideração o foco nas palavras mais importantes de toda a sequência.

Essa pontuação é dividida pela raiz quadrada do comprimento dos vetores k ($\sqrt{d^k}$) para garantir maior estabilidade dos gradientes (Figura 11 (g)). Com isso, o resultado de cada par de palavras é submetido à transformação *softmax*, que gera valores no intervalo entre 0 e 1, sendo que a soma de todos os valores é igual a 1 (Figura 11 (h)). Esse resultado representa a “atenção” que a rede deve dar a cada par de palavras. Em seguida, o valor do *softmax* é multiplicado pelo vetor v das palavras correspondentes (ou seja, a palavra com a qual a palavra-foco faz par), resultando em vetores s (Figura 11 (i)), que são somados, produzindo, finalmente, um vetor z para a palavra-foco (Figura 11 (j)), que incorpora as relações entre a palavra-foco e todas as

Figura 11 – Exemplo do cálculo de uma camada de *self-attention*.

Fonte: Própria (2024).

demais palavras da sequência, com a devida atenção considerada (Vaswani et al., 2017).

Todos os valores da Figura 11 são ilustrativos; porém, os vetores s e z estão em diferentes tons para demonstrar que, após a multiplicação dos valores pelo *softmax*, o valor de uma palavra terá maior impacto que o de outra, fazendo com que o vetor z de uma tenha maior peso que o de seu par (neste exemplo, o z_1), demonstrando a relação entre as palavras.

O módulo *encoder* possui um conjunto de matrizes W^q , W^k e W^v , e esse conjunto é denominado de *attention head* (ou cabeça de atenção). A quantidade de *attention heads* é a raiz quadrada do comprimento dos vetores k ($\sqrt{d^k}$). Com isso, o *transformer* realiza o mecanismo de atenção mais de uma vez, resultando, no final da camada de *self-attention*, em $n(\sqrt{d^k})$ vetores z para cada texto, fazendo com que haja atenção em diversos dados (Popel; Bojar, 2018). Esse processo é similar ao que ocorre com o ser humano, que, por exemplo, ao ler um livro, dá atenção a diversas partes do texto ao mesmo tempo, como o nome de um personagem, o ambiente em que se situa, entre outros, e o *transformer* funciona de maneira análoga.

Por fim, a última etapa é concatenar todos os vetores z (*attention heads*) e multiplicá-los por uma matriz de peso W^o , que também é ajustada durante o treinamento do modelo. Cada palavra será representada por um vetor z com o mesmo tamanho do vetor v , e esses vetores passam por uma camada *feed forward*, que irá gerar os *embeddings finais*, os quais são enviados pelo *encoder* para o *decoder*.

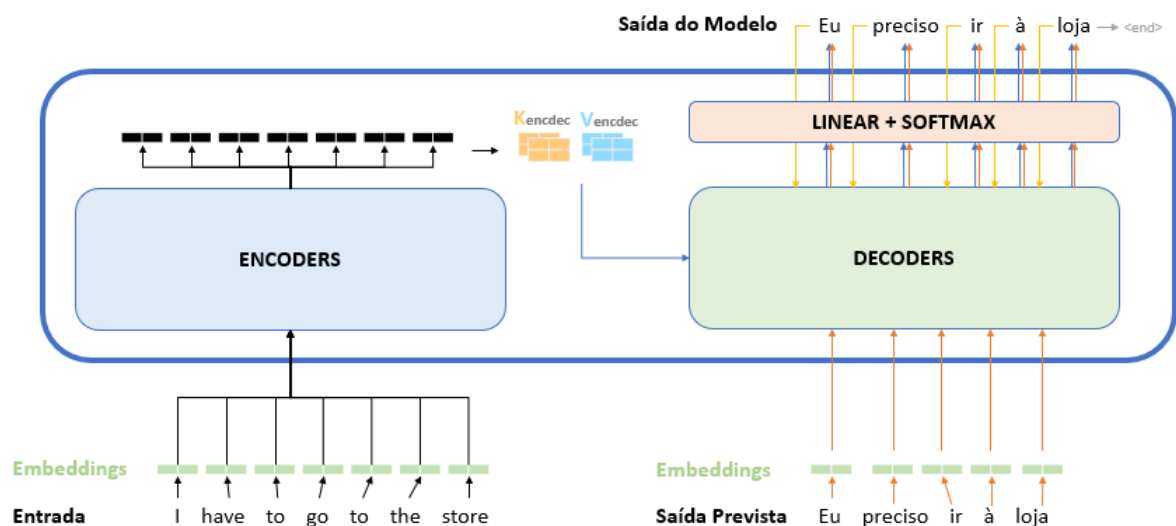
O *decoder* recebe os *embeddings* e transforma-os em matrizes de atenção K e V , que serão aplicadas nas camadas *encoder-decoder attention*. Essa camada aplica a camada *self-attention* no primeiro vetor que recebe, que, no exemplo de tradução, é o início da sequência traduzida, e esse processamento é o mesmo da camada *self-attention* do *encoder*. No entanto, na camada

encoder-decoder attention, são utilizadas as matrizes K e V (que são produtos do processamento da sequência de entrada), ao invés de utilizar as matrizes W^k e W^V .

O resultado da pilha que forma o *decoder* é, da mesma forma que no *encoder*, um *embedding* de comprimento igual ao *embedding* que representa o texto original. Esse *embedding* passa por uma camada linear, convertendo-o em um vetor que, no exemplo de tradução, tem comprimento igual ao do vocabulário do segundo idioma (português), resultando na representação de cada uma das palavras traduzidas.

Esse vetor é ativado pela função *softmax*, resultando em um vetor de probabilidades, e o valor com maior probabilidade indica a palavra que o modelo deve gerar, produzindo, finalmente, uma saída (Figura 12 (seta azul)). Ao retornar essa saída como resultado, a palavra volta para o início do *decoder* (Figura 12 (seta amarela)), para ser utilizada na previsão da próxima palavra, até atingir a última, gerando um *token* que indica que o texto terminou.

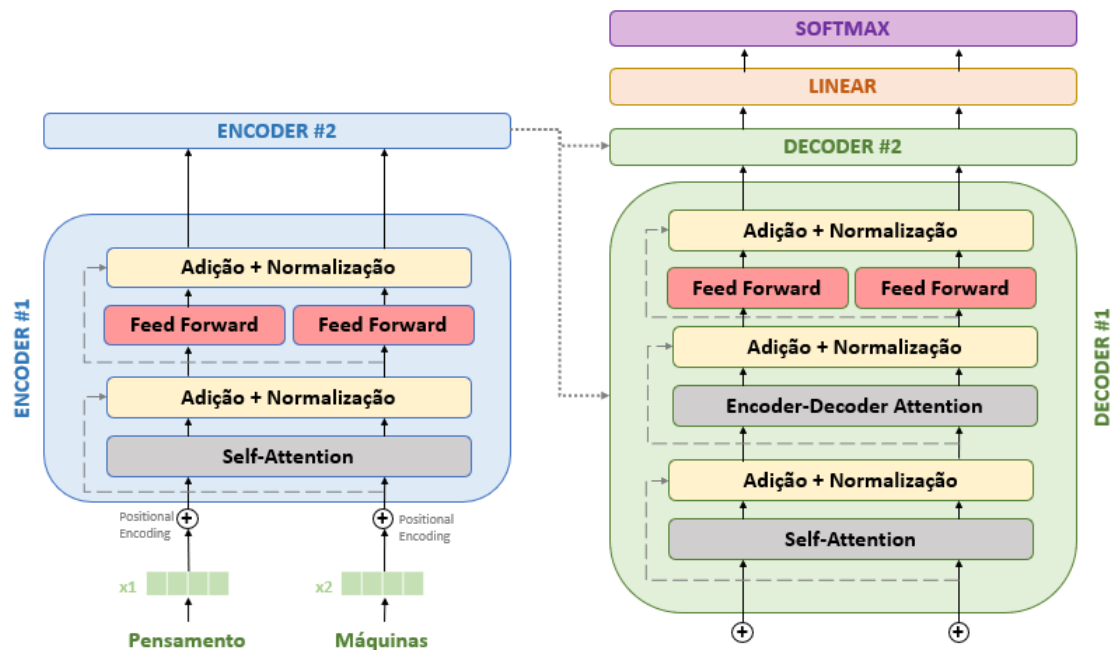
Figura 12 – Representação da pilha (*stack*) do *transformer*.



Fonte: Adaptado de (Alammar, 2018).

Com isso, a arquitetura *transformer* acaba sendo preferida em diversas tarefas em relação a RNNs e LSTMs, pois as palavras não precisam entrar na rede na sequência original; todas elas são processadas simultaneamente, permitindo a paralelização das operações, o que gera um ganho de performance em termos de velocidade. O mecanismo de atenção é aplicado de forma eficiente, criando uma relação entre as palavras, até mesmo aquelas distantes umas das outras, melhorando a performance no quesito precisão (Popel; Bojar, 2018).

Além disso, após cada unidade de atenção e feed-forward, existem camadas de adição e normalização (Popel; Bojar, 2018), e o *transformer* adiciona a cada *embedding* um *positional encoding*, que tem o objetivo de manter a relação temporal entre as palavras, ou seja, a rede sabe qual é a sequência original de cada palavra (Vaswani et al., 2017). A Figura 13 representa as camadas do *encoder* e *decoder* de uma pilha da arquitetura *transformer*.

Figura 13 – Camadas do *encoder* e *decoder* do *transformer*.

Fonte: Adaptado de (Alammar, 2018).

2.8 LLM

Grandes Modelos de Linguagem (*Large Language Models* ou LLMs) são modelos de inteligência artificial que utilizam técnicas de aprendizado de máquina para entender e gerar linguagem humana. Esses modelos se baseiam em redes neurais profundas e adotam técnicas de processamento de linguagem natural para computar e calcular suas respostas (TOUVRON et al., 2023).

As LLMs utilizam um método denominado aprendizado não supervisionado. Nesse processo, o modelo é alimentado com um grande conjunto de dados (bilhões de palavras e frases) com o objetivo de "aprender" sobre a linguagem (TOUVRON et al., 2023).

À medida que o modelo aprende os padrões de associação entre palavras, ele consegue prever estruturas de frases com base na probabilidade. O resultado final é um modelo capaz de capturar as complexas relações entre palavras e frases (TOUVRON et al., 2023).

Com um vasto conjunto de dados e numerosos cálculos probabilísticos, os LLMs exigem um volume significativo de recursos computacionais. As unidades de processamento gráfico (GPUs) são hardwares criados para lidar com tarefas de processamento complexas e simultâneas, sendo ideais para modelos como os LLMs (STRUBELL; GANESH; MCCALLUM, 2019).

Para aprimorar a capacidade de um LLM, a arquitetura Transformer absorve eficientemente as dependências e relações contextuais entre os elementos de uma sequência de dados através de seus parâmetros. Esses parâmetros definem limites que são essenciais para analisar o grande

volume de dados (NAVEED et al., 2024).

Com o desenvolvimento do ChatGPT e do GPT-4 (*Generative Pre-trained Transformer*) pela OpenAI, ocorreu uma revolução na percepção sobre os LLMs (OPENAI et al., 2024). Apesar de possuírem desempenho notável, a OpenAI não divulgou detalhes sobre as estratégias de treinamento ou os parâmetros de peso utilizados.

Em resposta, a Meta desenvolveu o modelo denominado LLaMA (*Large Language Model Meta AI*), que é de código aberto e possui tamanhos variados, de 7 bilhões até 405 bilhões de parâmetros. Os autores afirmam que o modelo apresenta desempenho equivalente aos principais modelos, como o GPT-4, e está próximo de atingir o estado da arte em uma variedade de tarefas (TOUVRON et al., 2023). O LLaMA consiste em duas etapas principais:

- **Pré-treinamento:** no qual o modelo é treinado em grande escala usando tarefas simples como a previsão da próxima palavra (AI@META, 2024).
- **Pós-treinamento:** no qual o modelo é ajustado para seguir instruções, alinhar-se às preferências humanas e melhorar capacidades específicas (como a codificação) (AI@META, 2024).

O modelo LLaMA, atualmente na versão 3.3, suporta nativamente multilinguismo, codificação e raciocínio. Seu maior modelo Transformer possui 405B de parâmetros e processa informações em uma janela de contexto de até 128k tokens (AI@META, 2024).

Para obter esses resultados, os autores afirmam que um grande modelo de linguagem de alta qualidade precisa de:

- **Dados:** Qualidade e quantidade de dados utilizados tanto no pré-treinamento quanto no pós-treinamento são essenciais. Foi utilizado um corpus de cerca de 15T de tokens multilíngues (AI@META, 2024).
- **Escala:** O modelo foi pré-treinado utilizando $3,8 \times 10^{25}$ FLOPs no pré-treinamento. FLOPs ou Operações de Ponto Flutuante representam o número de cálculos de ponto flutuante (adições, multiplicações, etc.) realizados durante o treinamento. A estimativa de FLOPs foi desenvolvida pela OpenAI: $FLOPs = 6ND$, onde N são parâmetros e D são tokens. O LLaMA 3 possui 400B de parâmetros, resultando em $6 \times 400 \times 15T = 3,8 \times 10^{25}$ (AI@META, 2024).
- **Gerenciamento da complexidade:** Maximizar a capacidade de escalar o processo de desenvolvimento do modelo utilizando o modelo Transformer padrão (Vaswani et al., 2017), além de adotar um procedimento de pós-treinamento simples com base em ajuste fino supervisionado (SFT), amostragem de rejeição (RS) e otimização de preferência direta (DPO) (RAFAILOV et al., 2024).

Para a criação do modelo LLaMA 3, o corpus multilíngue foi convertido em tokens e pré-treinado para executar a previsão do próximo token. Nesse estágio, o modelo aprende a estrutura da linguagem (AI@META, 2024).

O pré-treinamento é realizado em grande escala, utilizando um modelo de 405B de parâmetros em 15,6T de tokens, com uma janela de contexto inicial de 8k tokens. Esse processo é sequencial, aumentando progressivamente a janela de contexto até atingir 128k tokens. O modelo utiliza o algoritmo AdamW com uma taxa de aprendizado de pico de 8×10^{-5} , aquecimento linear de 8000 passos e uma programação de taxa de aprendizado decrescente em cosseno até 8×10^{-7} (AI@META, 2024).

Os dados do pré-treinamento incluem informações até o final de 2023 obtidas na web. Foram utilizados métodos de deduplicação e mecanismos de limpeza de dados em cada fonte para garantir tokens de alta qualidade, além de remover domínios com conteúdo adulto e informações pessoalmente identificáveis (AI@META, 2024).

Um classificador foi desenvolvido para categorizar os tipos de informações contidas nos dados, permitindo a inclusão de categorias específicas. Após vários testes, foi estabelecida a seguinte distribuição de tokens: 50% de conhecimento geral, 25% matemáticos e de raciocínio, 17% de código e 8% multilíngues (AI@META, 2024).

No estágio final do pré-treinamento, sequências longas foram utilizadas para dar suporte às janelas de contexto de até 128k tokens. O treinamento em sequências longas foi evitado inicialmente devido ao aumento quadrático na complexidade computacional das camadas de autoatenção. O comprimento do contexto foi aumentado gradualmente até que o modelo se adaptasse com sucesso (AI@META, 2024).

Este processo foi realizado em seis estágios, utilizando aproximadamente 800B de tokens de treinamento. Assim como a OpenAI afirmou, a Meta também relatou que o recozimento melhorou o desempenho dos modelos em até 24%, embora em algumas versões a melhoria tenha sido insignificante. Os autores afirmam que modelos com forte capacidade de aprendizado e raciocínio em contexto não necessitam de amostras específicas de treinamento em domínios para obter bom desempenho (AI@META, 2024).

O LLaMA 3 utiliza a arquitetura Transformer densa e padrão (Vaswani et al., 2017), com atenção de consulta agrupada (AINSLIE et al., 2023) e oito cabeças de chave-valor para melhorar a velocidade de inferência e reduzir o tamanho dos caches durante a decodificação. O modelo também emprega uma máscara de atenção que impede a autoatenção entre diferentes documentos dentro da mesma sequência.

O modelo LLaMA 3 de 405B de parâmetros possui uma arquitetura com 126 camadas e 128 cabeças de atenção, atingindo o tamanho computacionalmente ótimo de acordo com as leis de escala para o orçamento de treinamento da Meta. Essas leis determinam o tamanho ideal do modelo conforme o orçamento computacional (AI@META, 2024).

Após o modelo adquirir uma rica compreensão da linguagem, mas ainda não ser capaz de seguir instruções, inicia-se o pós-treinamento. Este estágio utiliza um modelo de recompensa sobre o ponto de verificação pré-treinado com dados de preferência anotados por humanos. Em seguida, os pontos de verificação são ajustados com Ajuste Fino Supervisionado (SFT) em dados de ajuste de instrução e Otimização de Preferência Direta (DPO) (AI@META, 2024).

O Ajuste Fino Supervisionado (SFT) é uma etapa de treinamento em modelos AM, especialmente usada em LLMs. Essa etapa ocorre após o treinamento inicial (pré-treinamento) do modelo em um grande conjunto de dados não supervisionados. No SFT, o modelo é ajustado usando um conjunto de dados rotulado e supervisionado, onde há exemplos de entrada e saída esperada. O objetivo é alinhar o modelo com tarefas específicas, reduzindo erros e melhorando seu desempenho em problemas delimitados.

A Otimização de Preferência Direta (DPO) é uma técnica emergente que busca alinhar os modelos diretamente com preferências humanas ou critérios de qualidade especificados. Em vez de ajustar o modelo com base em dados supervisionados ou respostas específicas, a DPO utiliza comparações de preferências humanas (como exemplo, a DPO retornará "Resposta A é melhor que Resposta B"). Com base nessas comparações, o modelo é ajustado para produzir respostas que maximizem a preferência do usuário.

O Modelo de Recompensa (RM) utiliza dados de preferência para gerar pares de respostas (escolhida e rejeitada), além de gerar anotações para edição de respostas. Cada amostra de classificação de preferência possui duas ou três respostas com hierarquia clara (editado > escolhido > rejeitado) (AI@META, 2024).

Por fim, o prompt é concatenado com as respostas em uma única linha, embaralhadas aleatoriamente, e então separadas para cálculo de pontuação. Esta abordagem melhora a eficiência do treinamento sem perda de precisão.

Dentre as técnicas para especializar LLMs em tarefas de domínio específico sem alterar seus pesos, destaca-se a *Engenharia de Prompt*, que envolve a preparação cuidadosa das instruções enviadas ao modelo para obter respostas características, concisas e contextualmente adequadas. A Engenharia de Prompt tornou-se uma prática essencial para o aproveitamento máximo das potencialidades dos LLMs (BROWN et al., 2020), sendo a abordagem adotada nesta pesquisa.

O *prompt* funciona como uma configuração dos modelos para que uma tarefa possa ser concluída. Não se trata apenas da criação de instruções independentes, mas de uma habilidade que tenha um entrosamento profundo das capacidades e limitações dos LLMs. Existem vários estágios para refinar, repetir e testar continuamente os prompts com base nos resultados obtidos (NASCIMENTO, 2024).

O *prompt* deve ser descrito de forma clara e precisa, contendo as informações necessárias para que o modelo compreenda a tarefa a ser executada, bem como o formato da resposta esperada. Para alcançar melhor qualidade nos dados extraídos, Zhang afirma que, para modelos LLaMa,

os prompts devem ser mantidos pequenos e simples para um melhor entendimento da tarefa (ZHANG et al., 2024).

Prompts muito longos podem fazer com que os modelos percam informações relevantes. Além disso, a quantidade de amostras fornecidas como exemplos influencia diretamente na qualidade da extração, pois modelos com mais exemplos claros possibilitam uma compreensão mais precisa de como a tarefa deve ser realizada (ZHANG et al., 2024).

2.9 RASE

Com o objetivo de captar informações presentes em regulamentos, estruturar o processo de conversão dessas regras para uma linguagem computacional e aplicar a verificação automática de modelos em grande escala, Hjelseth desenvolveu as metodologias Tx3 e RASE (HJELSETH, 2015).

A metodologia Tx3 propõe três ações a serem realizadas conforme a classificação das informações presentes nos regulamentos:

- **Transcrever (T1):** Deve ser aplicada para instruções que podem ser diretamente transcritas em regras computáveis;
- **Transformar (T2):** Deve ser aplicada para instruções que necessitam ser reestruturadas ou reescritas de forma a preservar o sentido original;
- **Transferir (T3):** Deve ser aplicada quando os requisitos são expressos de maneira imprecisa, sem conexão clara entre os objetivos do regulamento e os indicadores. Nestes casos, é necessária a análise de um profissional especializado.

Após a classificação do texto original, os elementos classificados como T1 e T2 são reescritos e submetidos à metodologia RASE. A metodologia RASE, baseada em conceitos semânticos, transforma documentos normativos em regras bem definidas, utilizáveis em softwares baseados em BIM (HJELSETH; NISBET, 2011). O processo consiste na identificação de quatro operadores lógicos:

- **Requisito (R):** Expressa o dever associado, frequentemente identificado pelo imperativo “deve”. Por exemplo, na frase “Toda rota acessível de espaço privado deve ser provida de iluminação natural ou artificial com nível mínimo de iluminância de 150 lux medidos a 1,00 m do chão”, o requisito é: “deve ser provida de iluminação natural ou artificial com nível mínimo de iluminância de 150 lux”.
- **Aplicabilidade (A):** Identifica a quem ou ao que o requisito se aplica. No exemplo anterior, a aplicabilidade é: “rota acessível”.

- **Seleção (S):** Indica um subconjunto da aplicabilidade. No exemplo, a seleção é: “de espaço privado”;
- **Exceção (E):** Define os elementos excluídos da regra. Completando o exemplo anterior com a norma “São aceitos níveis inferiores de iluminância para ambientes específicos, como cinemas, teatros ou outros, conforme normas técnicas específicas.”, a exceção é: “níveis inferiores para cinemas, teatros e outros”, pois isso não se aplica a regra.

Para facilitar a visualização, utiliza-se um sistema de cores, onde de costume é utilizado: Requisitos (azul), Aplicabilidade (verde), Seleção (vermelho) e Exceção (laranja). Cada operador identificado é associado a atributos específicos:

- **Objeto:** Termos padronizados, como “escada” ou “corrimão”.
- **Propriedade (A):** Termos classificados, como “nível de iluminância” ou “sistema de segurança”.
- **Comparador:** Pode ser maior, menor, igual, diferente, entre outros.
- **Valor:** Valores numéricos, unidades ou descrições.
- **Unidade:** Tipo de unidade do valor numérico caso exista.

Como exemplo, considere o texto da norma NBR 9050: “A inclinação transversal da superfície deve ser de até 2% para pisos internos e de até 3% para pisos externos. A inclinação longitudinal da superfície deve ser inferior a 5%. Inclinações iguais ou superiores a 5% são consideradas rampas e, portanto, devem atender a 6.6.”

Aplicando a metodologia Tx3 e RASE:

- **Tx3 (Transcrever):**
 - Pisos internos devem ter inclinação transversal menor ou igual a 2%;
 - Pisos externos devem ter inclinação transversal menor ou igual a 3%;
 - Pisos internos devem ter inclinação longitudinal menor do que 5%;
 - Pisos externos devem ter inclinação longitudinal menor do que 5%;
 - Pisos com inclinação longitudinal maior ou igual a 5% são considerados rampas e devem atender a 6.6;
- **RASE (Operadores):**
 - A{Pisos} S{internos} R{devem ter inclinação transversal menor ou igual a 2%};
 - A{Pisos} S{externos} R{devem ter inclinação transversal menor ou igual a 3%};

- A{Pisos} S{internos} R{devem ter inclinação longitudinal menor do que 5%};
- A{Pisos} S{externos} R{devem ter inclinação longitudinal menor do que 5%};
- A{Pisos} E{com inclinação longitudinal maior ou igual a 5% são consideradas rampas} R{devem atender a 6.6};

Por fim, após obter os operadores RASE é gerado o atributo de cada um dos operadores, como mostra a Figura 14.

Figura 14 – Utilização da metodologia RASE na NBR 9050

	Frase Métrica	Operador	Objeto	Propriedade	Comparador	Valor	Und.
Pisos internos devem ter inclinação transversal menor ou igual a 2%.	Pisos	aplicabilidade	componente	tipo	inclui	piso	
	internos	seleção	piso	interno	=	VERDADEIRO	
	devem ter inclinação transversal menor ou igual a 2%.	requisito	piso interno	inclinação transversal	<=	2	%
Pisos externos devem ter inclinação transversal menor ou igual a 3%.	Pisos	aplicabilidade	componente	tipo	inclui	piso	
	externos	seleção	piso	externo	=	FALSO	
	devem ter inclinação transversal menor ou igual a 3%.	requisito	piso externo	inclinação transversal	<=	3	%
Pisos internos e externos devem ter inclinação longitudinal menor do que 5%.	Pisos	aplicabilidade	componente	tipo	inclui	piso	
	internos	seleção	piso	interno	=	VERDADEIRO	
	devem ter inclinação longitudinal menor do que 5%.	requisito	piso interno	inclinação longitudinal	<	5	%
Pisos internos e externos devem ter inclinação longitudinal menor do que 5%.	Pisos	aplicabilidade	componente	tipo	inclui	piso	
	externos	seleção	piso	externo	=	FALSO	
	devem ter inclinação longitudinal menor do que 5%.	requisito	piso externo	inclinação longitudinal	<	5	%
Pisos com inclinação longitudinal maior ou igual a 5% são consideradas rampas devem atender a 6.6.	Pisos	aplicabilidade	componente	tipo	inclui	piso	
	com inclinação longitudinal maior ou igual a 5% são consideradas rampas	exceção	piso	inclinação longitudinal	>=	5	%
	devem atender a 6.6.	requisito	rampa	atender	inclui	Item 6.6 RAMPAS	

Fonte: Própria (2024).

2.10 Trabalhos Relacionados

A integração do BIM com tecnologias de inteligência artificial generativa (IAG), como o ChatGPT, tem sido pouco explorada no setor de Arquitetura, Engenharia e Construção (AEC). Este tema abrange aplicações práticas, frameworks inovadores, desafios e possibilidades futuras, sendo objeto de estudo de diversos trabalhos recentes.

Um dos estudos analisados aborda a integração do BIM com modelos de IAG, como o ChatGPT e o Bard. Este artigo identifica que a utilização dessas tecnologias visa otimizar processos como design e tomada de decisão (RANE; CHOUDHARY; RANE, 2023). No entanto, os autores destacam desafios, como a falta de padronização de dados, deficiências em segurança e a necessidade de capacitação de profissionais do setor (RANE; CHOUDHARY; RANE, 2023). Além disso, os modelos generativos enfrentam dificuldades em interpretar a semântica complexa de dados BIM, além de problemas gerais, como alta demanda computacional e falta de diretrizes éticas (RANE; CHOUDHARY; RANE, 2023). Em resposta a essas lacunas, os pesquisadores propõem um framework voltado à otimização de processos, redução de custos e tempo, e à facilitação da adoção de IA por meio de interfaces mais intuitivas (RANE; CHOUDHARY; RANE, 2023).

Outro estudo relevante trata do desenvolvimento de um chatbot, denominado JULE, projetado para gerentes de obras (LIN, 2023). Este modelo utiliza BIM e técnicas de Processamento de Linguagem Natural (PLN) para fornecer informações precisas e em tempo real no canteiro de obras. O chatbot permite que os usuários consultem dados como dimensões de componentes estruturais e ferramentas necessárias de forma ágil e intuitiva, eliminando a dependência de interfaces convencionais (LIN, 2023). A abordagem proposta demonstrou alta precisão na identificação de intenções e na recuperação de informações essenciais, melhorando a adoção de sistemas baseados em BIM e promovendo maior eficiência no gerenciamento de obras (LIN, 2023).

Um trabalho adicional investigou a aplicação de LLMs, como GPT-3.5 e GPT-4, em projetos BIM voltados à segurança (SAMMOUR et al., 2024). Os autores realizaram testes com 385 questões de múltipla escolha de exames certificados pela BCSP, abrangendo áreas como gestão de sistemas de segurança, identificação de perigos e ergonomia (SAMMOUR et al., 2024). Utilizando técnicas de engenharia de *prompt*, como *Prompt Direct*, *Prompt Chain-of-Thought* e *Prompt Few-Shot*, os modelos foram avaliados em termos de precisão e consistência. Os resultados indicaram que o GPT-4 alcançou 84,6% de precisão, enquanto o GPT-3.5 obteve 73,8%. Apesar do bom desempenho em áreas como gerenciamento de sistemas e controle de perigos, ambos os modelos apresentaram dificuldades em responder questões relacionadas a emergências e cálculos matemáticos (SAMMOUR et al., 2024). A pesquisa destacou o potencial desses modelos, mas também a necessidade de melhorias para evitar informações imprecisas ou enviesadas (SAMMOUR et al., 2024).

Por fim, outro estudo explorou diferentes técnicas de engenharia de prompt para otimizar a utilização de LLMs no setor AEC. Os autores identificaram que métodos genéricos de interação limitam a eficácia desses modelos, resultando em saídas desalinhadas com as necessidades do setor (SAMSAMI, 2024). Problemas como confiabilidade, adaptação a regulamentações e aceitação no setor também foram identificados como barreiras. A pesquisa coletou técnicas de prompt eficazes, como prompts instrucionais e baseados em restrições, e testou essas abordagens em projetos reais (SAMSAMI, 2024). Os resultados mostraram que prompts específicos reduziram erros em cronogramas e melhoraram o reconhecimento de perigos. Apesar disso, limitações como alucinações do modelo para tarefas complexas e a necessidade de validação externa ainda persistem (SAMSAMI, 2024).

Os trabalhos analisados fornecem uma visão abrangente sobre o estado da arte na integração de LLMs e BIM no setor AEC. Embora avanços significativos tenham sido alcançados, desafios importantes, como a interpretação semântica de dados BIM, segurança e a adaptação de LLMs às demandas específicas do setor, ainda necessitam de soluções mais robustas. Assim, este campo continua a ser promissor para futuras pesquisas e inovações.

2.10.1 Análise dos Trabalhos Relacionados

A integração de BIM com LLMs e IA generativa no setor AEC tem mostrado avanços, mas também apresenta desafios. Um estudo identificou a utilização de tecnologias como ChatGPT e Bard para otimizar processos, como design e tomada de decisão. No entanto, as dificuldades incluem a falta de padronização de dados, questões de segurança e a complexidade semântica dos dados BIM (RANE; CHOUDHARY; RANE, 2023). O uso de LLMs aqui é bom devido à sua capacidade de processar grandes volumes de dados e fornecer *insights* valiosos, mas a interpretação semântica continua sendo uma limitação.

Outro estudo sobre o chatbot JULE, que usa BIM e PLN, mostrou avanços na eficiência do gerenciamento de obras, permitindo consultas rápidas no canteiro. No entanto, como o modelo depende de dados BIM bem estruturados, as falhas na semântica e na adaptação dos dados para o modelo ainda representam uma barreira (LIN, 2023). Mesmo assim, o uso de LLMs para interpretar regras e padrões regulatórios é promissor devido à sua agilidade na recuperação de informações.

Um estudo com LLMs como GPT-3.5 e GPT-4 aplicados a segurança em BIM demonstrou precisão significativa, mas também dificuldades em questões complexas como emergências e cálculos. A limitação aqui está na falta de confiabilidade para questões mais técnicas, e embora os LLMs possam interpretar normas e regulamentos, é necessário garantir maior precisão e evitar informações enviesadas (SAMMOUR et al., 2024).

Além disso, a pesquisa sobre técnicas de engenharia de prompt para otimizar LLMs no setor AEC destacou que prompts genéricos não são eficazes. O uso de prompts mais específicos

melhorou a precisão, mas problemas como alucinações do modelo e a necessidade de validação externa persistem (SAMSAMI, 2024). A adaptação dos modelos às regulamentações específicas do setor é crucial, e o uso de LLMs para interpretar regras regulatórias se mostra útil, mas ainda exige ajustes finos e validação.

Diferencial desta pesquisa. Os trabalhos revisados concentram-se em LLMs proprietários (ChatGPT, GPT-3.5, GPT-4, Bard) e em aplicações pontuais de Engenharia de Prompt, sem comparações sistemáticas entre modelos *open-source* nem decomposição explícita da tarefa em níveis. Esta dissertação distingue-se ao avaliar comparativamente seis LLMs *open-source* servidos localmente via Ollama (Llama, Dolphin, Gemma, Mistral, Alpaca e Qwen), aplicando-os à conversão de normas brasileiras da AEC para o formato RASE em três níveis sucessivos de normalização (N1, N2 e N3) e em três experimentos (EN1, EN2 e EN1N2). A avaliação combina métricas lexicais, semânticas e de distância, complementadas por métricas de classificação, oferecendo um retrato multidimensional do desempenho de cada modelo.

3

Metodologia

Este capítulo descreve a metodologia adotada na pesquisa, organizada em torno de três decisões centrais: (i) a decomposição da tarefa de conversão de normas para o formato RASE em três níveis sucessivos de normalização (N1, N2 e N3); (ii) o uso exclusivo de Engenharia de Prompt como técnica de especialização dos modelos, sem *Fine-Tuning* e sem *Retrieval-Augmented Generation*; e (iii) a comparação sistemática de seis Modelos de Linguagem de Grande Porte (LLMs) *open-source* servidos localmente via Ollama. O capítulo apresenta também o *dataset* utilizado, o pipeline de execução, os cinco experimentos conduzidos (EN1, EN2, EN1N2, EN3 e EN1N2N3), as métricas de validação e o ambiente computacional empregado.

3.1 Visão Geral

A pesquisa avaliou o uso de LLMs *open-source* para converter normas técnicas da AEC em representações computáveis no formato RASE, utilizando **exclusivamente Engenharia de Prompt**. A conversão foi modelada como um processo de normalização em três níveis sucessivos: N1 (segmentação atômica das sentenças normativas), N2 (identificação dos operadores RASE) e N3 (estruturação semântica em JSON). Seis modelos LLM foram avaliados sob cinco experimentos (EN1, EN2, EN1N2, EN3 e EN1N2N3), aplicados a um *dataset* composto por 79 normas brasileiras anotadas, e comparados por meio de métricas de similaridade textual e de classificação.

3.2 Necessidade de Normalização em Três Níveis

A transformação completa de uma norma técnica em texto bruto para a sua representação RASE final em JSON envolve raciocínios distintos: segmentação sintática, classificação semântica e extração de atributos estruturados. Pedir todas essas decisões em uma única chamada ao LLM tende a degradar a qualidade da saída, induzindo alucinações, omissões e estruturas inválidas. A

decomposição em três níveis sucessivos isola cada decisão, mantém cada chamada ao modelo com escopo bem delimitado e permite validar cada etapa independentemente.

3.2.1 N1 — Segmentação Atômica

Uma sentença normativa frequentemente contém múltiplas regras coordenadas. Por exemplo, o texto “A inclinação transversal deve ser de até 2% para pisos internos e de até 3% para pisos externos” contém duas regras computáveis distintas. O nível N1 tem como objetivo segmentar a sentença original em sentenças atômicas, em que cada item representa uma única regra. Sem essa segmentação, as etapas subsequentes não conseguem aplicar corretamente os operadores RASE.

O prompt utilizado em N1 é mantido em um arquivo de texto dedicado a este nível e instrui o modelo a quebrar a sentença em sentenças curtas e diretas, uma regra computável por sentença, sem inventar elementos, acompanhado de um exemplo demonstrativo.

3.2.2 N2 — Identificação dos Operadores RASE

Dada uma regra atômica produzida em N1, o nível N2 tem como objetivo identificar quais trechos correspondem a Requisito (R), Aplicabilidade (A), Seleção (S) e Exceção (E), segundo a metodologia RASE. Essa classificação é o cerne da formalização: sem ela, o texto permanece em linguagem natural.

O prompt N2 é armazenado em um arquivo de texto próprio e define uma ordem fixa para os quatro operadores e um formato exato de saída em quatro linhas, marcando o campo vazio com aspas duplas (""). O prompt inclui um exemplo único embutido, que serve de âncora para o modelo.

3.2.3 N3 — Estruturação Semântica em JSON

Mesmo após classificar um trecho como R, A, S ou E, o conteúdo permanece em linguagem natural. Para que uma regra possa ser verificada automaticamente contra um modelo BIM, é necessário extrair um tripé estruturado composto por *objeto*, *propriedade* e *valor*, complementado por *comparador* e *unidade*. O nível N3 produz esta representação em formato JSON, com os campos `type`, `object`, `property`, `comparison`, `target` e `unit`.

Em função da natureza distinta de cada operador RASE, optou-se por utilizar **um prompt por operador** em N3, com arquivos de texto separados para Aplicabilidade, Requisito, Seleção e Exceção. A Aplicabilidade descreve *a quem* a regra se aplica; o Requisito descreve *o que é* exigido; a Seleção restringe o escopo; e a Exceção define os casos não cobertos. Cada prompt traz o esquema JSON explícito, regras heurísticas de mapeamento por padrões textuais comuns (por exemplo, “uso público” → `property "uso"`, `comparison "="`, `target "público"`) e múltiplos exemplos *few-shot* por operador.

3.3 Engenharia de Prompt

A pesquisa utilizou exclusivamente Engenharia de Prompt como abordagem de especialização. Não foi empregado *Fine-Tuning* dos modelos, nem *Retrieval-Augmented Generation*, nem qualquer banco vetorial. Toda a adaptação dos LLMs à tarefa foi realizada pelo desenho dos prompts, organizados em arquivos de texto dedicados a cada nível e a cada operador. Três técnicas clássicas de Engenharia de Prompt foram aplicadas, cada uma escolhida em função da natureza do nível em que atua.

3.3.1 Prompt Direct

O *Prompt Direct* consiste em uma instrução direta e objetiva, sem encadeamento explícito de raciocínio. Foi aplicado ao nível **N1**, cuja tarefa — segmentar uma sentença em sub-regras — é estruturalmente simples e não exige raciocínio profundo sobre semântica normativa. Suas principais vantagens são o menor consumo de tokens e a menor latência; o risco principal é a queda de robustez diante de sentenças longas ou ambíguas. O prompt N1 consiste em cinco regras imperativas e um exemplo curto, sem instruções de raciocínio passo a passo.

3.3.2 Prompt Chain-of-Thought

O *Prompt Chain-of-Thought* instrui o modelo a explicitar o raciocínio em etapas antes da resposta final. Foi aplicado ao nível **N2**, em que a classificação de trechos como R, A, S ou E exige a interpretação do papel sintático-semântico do trecho dentro da regra. Esta técnica reduz alucinações e melhora a classificação em sentenças com múltiplos operadores, ao custo de maior consumo de tokens. O prompt N2 apresenta a ordem fixa dos passos (1. aplicabilidade → 2. seleção → 3. exceção → 4. requisito), guiando o modelo por uma cadeia de decisão explícita.

3.3.3 Prompt Few-Shot

O *Prompt Few-Shot* inclui no próprio prompt exemplos completos de entrada e saída, ancorando o formato esperado. Foi aplicado ao nível **N3**, em que a saída precisa obedecer a um esquema JSON rígido: sem exemplos, os modelos divergem do formato ou inventam campos. A técnica traz alta aderência ao formato e à terminologia esperada, ao custo de consumir parte da janela de contexto com os exemplos. Cada um dos prompts N3, um por operador RASE, traz de dois a quatro exemplos completos no padrão *operador N2 → JSON*.

3.4 Modelos LLM Avaliados

Seis modelos *open-source* foram servidos localmente por meio do **Ollama** (ollama serve), conforme a Tabela 1. Todos foram executados sobre o mesmo hardware, com os mesmos prompts e o mesmo *dataset*, garantindo comparabilidade direta entre os resultados.

Tabela 1 – Modelos LLM *open-source* avaliados.

Modelo	Origem	Notas
Llama	Meta	Modelo de referência, maior latência observada
Dolphin	Cognitive Computations	Variante alinhada por instruções
Gemma	Google	Modelo compacto
Mistral	Mistral AI	LLM eficiente em parâmetros
Alpaca	Stanford	Variante Llama instruída
Qwen	Alibaba	LLM multilíngue

3.5 Dataset

O *dataset* utilizado foi consolidado em um único arquivo JSON, com aproximadamente 270 KB e **79 normas** anotadas. As normas foram derivadas de regulamentações brasileiras da AEC, com ênfase na NBR 9050 ([ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2020](#)) (acessibilidade a edificações, mobiliário, espaços e equipamentos urbanos), adaptadas a partir do trabalho de Eduardo ([MENDONÇA; MANZIONE, 2020](#)).

Cada entrada do *dataset* é estruturada da seguinte forma:

- **text**: texto bruto da norma.
- **texts_n1**: lista de regras atômicas, cada uma com **text_n1** e **operators_n2**.
- **operators_n2**: dicionário com as chaves **requirements**, **applicability**, **selection** e **exception**, cada uma com **text_n2** e **properties_n3**.
- **properties_n3**: representação em JSON com os campos **type**, **object**, **property**, **comparation**, **target** e **unit**.

O mesmo arquivo serve como entrada para os geradores e como *target* de referência para os validadores, simplificando a reprodutibilidade dos experimentos.

3.6 Pipeline de Execução e Experimentos

A orquestração dos experimentos é realizada por um *script* principal, que aciona um menu para a etapa de geração e outro para a etapa de validação. Para cada modelo e para cada experimento, as saídas geradas são persistidas em um arquivo JSON próprio, com o padrão de nomenclatura `generate_<nível>_<modelo>.json` (por exemplo, `generate_n1_alpaca.json` para o nível N1 produzido pelo modelo Alpaca). Os resultados das validações também são armazenados em arquivos JSON dedicados, um por experimento, no padrão `validate_<experimento>.json` (por exemplo, `validate_n1.json`, `validate_n1n2.json` e `validate_n1n2n3.json`).

Para isolar as contribuições de cada nível e avaliar a propagação de erros, foram definidos cinco experimentos.

3.6.1 Experimento EN1

O experimento EN1 avalia a geração N1 isoladamente. A entrada é o texto bruto da norma, e a saída gerada é comparada ao N1 de referência presente no *dataset*. O experimento mede a qualidade da segmentação a partir do texto bruto, sem influência de etapas posteriores.

3.6.2 Experimento EN2

O experimento EN2 avalia a geração N2 isoladamente, eliminando o ruído proveniente da etapa anterior. A entrada é o N1 de referência (extraído diretamente do *dataset*), e a saída gerada é comparada ao N2 de referência, por operador (R, A, S e E). O experimento mede a qualidade pura da classificação RASE.

3.6.3 Experimento EN1N2

O experimento EN1N2 corresponde ao pipeline encadeado dos dois primeiros níveis: o N1 gerado pelo modelo na primeira etapa alimenta a etapa N2 do mesmo modelo. A saída final é comparada ao N2 de referência. O objetivo é avaliar a **propagação de erros** entre os dois níveis em um cenário totalmente automatizado.

3.6.4 Experimento EN3

O experimento EN3 avalia a geração N3 isoladamente, eliminando o ruído proveniente das etapas anteriores. A entrada é composta pelo N2 de referência por operador (extraído diretamente do *dataset*), e a saída gerada é comparada à representação JSON estruturada de referência, considerando os campos `type`, `object`, `property`, `comparation`, `target` e `unit`. O experimento mede a qualidade pura da estruturação semântica em JSON, sem influência das etapas de segmentação ou de classificação.

3.6.5 Experimento EN1N2N3

O experimento EN1N2N3 corresponde ao pipeline encadeado completo: o N1 gerado pelo modelo alimenta a etapa N2, que por sua vez alimenta a etapa N3 do mesmo modelo. A saída final — uma representação em JSON estruturada por operador — é comparada à referência do *dataset*. O objetivo é avaliar a **propagação de erros** ao longo dos três níveis em um cenário totalmente automatizado, replicando o uso real do pipeline em produção.

3.7 Métricas de Validação

As saídas dos LLMs foram avaliadas por dez métricas, organizadas em quatro famílias: similaridade lexical, similaridade semântica, distância semântica e classificação.

3.7.1 Similaridade Lexical

- **FuzzyWuzzy**: utiliza `fuzz.partial_ratio`, baseado na distância de Levenshtein parcial; tolera fragmentos da string-alvo.
- **TF-IDF**: cosseno entre vetores TF-IDF das duas *strings*, ponderando termos pela sua importância no corpus.

3.7.2 Similaridade Semântica

- **SBERT (PT)**: *embeddings* produzidos pelo modelo `tgsc/sentence-transformer-ult5-pt-small`.
- **BERTimbau**: *embeddings* de `rufimelo/Legal-BERTimbau-sts-large-ma-v3`, especializado em texto jurídico em português brasileiro.
- **Multilingual**: *embeddings* de `sentence-transformers/paraphrase-multilingual-mpnet-base` treinado em mais de cinquenta idiomas.

3.7.3 Distância Semântica (WMD)

- **WMD_ft**: *Word Mover's Distance* com *embeddings* do FastText (`fasttext-wiki-news-subwords-300`).
- **WMD_nlc**: WMD com *embeddings* do projeto NILC (`cbow_s300.txt`), com normalização para o intervalo $[0, 1]$.

3.7.4 Métricas de Classificação

Complementarmente às métricas de similaridade, são calculadas quatro métricas de classificação, derivadas da matriz de confusão entre a saída do modelo e a referência. O critério de acerto adotado consiste em um limiar sobre a similaridade semântica (SBERT ou Multilingual) para os níveis N1 e N2, e em correspondência exata por campo do JSON para o nível N3.

- **Acurácia**: $ACC = (VP + VN) / (VP + VN + FP + FN)$.
- **Precisão**: $PRE = VP / (VP + FP)$.
- **Revocação**: $REC = VP / (VP + FN)$.
- **F1-score**: $F1 = 2 \cdot (PRE \cdot REC) / (PRE + REC)$.

As quatro novas métricas são persistidas no mesmo arquivo de validação de cada experimento, ao lado das métricas de similaridade.

3.8 Ambiente Computacional

Todas as execuções foram conduzidas em uma única estação de trabalho, com a seguinte configuração:

- **CPU:** Intel Core i9-13900K, 13ª geração, 3,00 GHz.
- **Memória RAM:** 32 GB.
- **GPU:** NVIDIA RTX 4080.
- **Sistema operacional:** Ubuntu 22.04.
- **Python:** 3.11.9.
- **Servidor LLM:** Ollama (`ollama serve`).
- **Bibliotecas principais:** `sentence-transformers`, `gensim`, `fuzzywuzzy` e `scikit-learn` (para TF-IDF).

A padronização do ambiente, dos prompts e do *dataset* entre os seis modelos garante que as diferenças observadas nos capítulos seguintes possam ser atribuídas ao comportamento dos LLMs e não a variações de infraestrutura ou de protocolo de avaliação.

4

Resultados

Este capítulo apresenta os resultados obtidos pela aplicação dos seis Modelos de Linguagem de Grande Porte (LLMs) *open-source* sobre as 79 normas brasileiras do *dataset*, nos cinco experimentos definidos no capítulo anterior: EN1 (segmentação N1 isolada), EN2 (identificação RASE com N1 de referência), EN1N2 (pipeline encadeado dos dois primeiros níveis), EN3 (estruturação JSON com N2 de referência) e EN1N2N3 (pipeline encadeado completo). Para cada experimento são reportadas as médias agregadas por métrica, os resultados detalhados por modelo, os tempos de execução observados e uma discussão integrada do desempenho. As métricas analisadas são as seis de similaridade textual (FuzzyWuzzy, TF-IDF, SBERT, Multilingual, WMD_ft e WMD_nilc), complementadas pela análise por modelo no nível BERTimbau apresentada ao final.

4.1 Visão Geral dos Experimentos

A Tabela 2 sintetiza as médias por métrica nos três experimentos voltados aos níveis N1 e N2, agregadas sobre os seis modelos. Observa-se a superioridade consistente das métricas baseadas em *embeddings* (SBERT, Multilingual e WMD), bem como a perda gradual de desempenho na transição de EN1 para EN1N2. Os experimentos EN3 e EN1N2N3, que avaliam a estruturação semântica em JSON, são reportados em seção própria mais adiante, por adotarem critério de acerto diferente (correspondência exata por campo) e por sintetizarem o desempenho do pipeline completo.

Tabela 2 – Médias por métrica nos três experimentos.

Experimento	FuzzyWuzzy	TF-IDF	SBERT	Multilingual	WMD_ft	WMD_nilc
EN1	0,638	0,460	0,710	0,749	0,661	0,659
EN2	0,528	0,462	0,587	0,668	0,694	0,655
EN1N2	0,502	0,360	0,567	0,650	0,628	0,608

4.2 Resultados por Experimento

4.2.1 EN1 — Segmentação N1

No experimento EN1, a comparação concentrou-se na proximidade entre as regras N1 geradas pelo LLM e as regras N1 de referência. As métricas semânticas superaram as lexicais (SBERT 0,710 e Multilingual 0,749 contra TF-IDF 0,460 e FuzzyWuzzy 0,638), indicando que a tarefa de segmentação tolera reformulações lexicais quando a semântica é preservada. O modelo **Dolphin** liderou as métricas semânticas e de distância (SBERT 0,840; Multilingual 0,873; WMD_ft 0,748; WMD_nilc 0,740), enquanto o **Gemma** obteve a maior pontuação em FuzzyWuzzy (0,724). A Tabela 3 apresenta os resultados detalhados por modelo.

Tabela 3 – Resultados por modelo em EN1.

Modelo	FuzzyWuzzy	TF-IDF	SBERT	Multilingual	WMD_ft	WMD_nilc
Alpaca	0,576	0,331	0,605	0,645	0,608	0,630
Dolphin	0,667	0,650	0,840	0,873	0,748	0,740
Gemma	0,724	0,550	0,765	0,789	0,687	0,681
Llama	0,711	0,535	0,764	0,788	0,674	0,665
Mistral	0,656	0,508	0,747	0,804	0,669	0,667
Qwen	0,491	0,189	0,538	0,596	0,580	0,569

4.2.2 EN2 — Identificação dos Operadores RASE

No experimento EN2, a análise concentrou-se na qualidade da identificação e estruturação dos operadores RASE a partir do N1 de referência. As médias agregadas foram inferiores às do EN1 (SBERT 0,587; Multilingual 0,668), evidenciando a maior dificuldade da formalização estruturada em comparação com a segmentação. As métricas de distância semântica (WMD_ft 0,694; WMD_nilc 0,655) superaram TF-IDF e FuzzyWuzzy, sinalizando robustez a variações de superfície. O modelo **Llama** apresentou liderança consistente em todas as métricas (Multilingual 0,869; WMD_ft 0,857; SBERT 0,806; FuzzyWuzzy 0,761; TF-IDF 0,777; WMD_nilc 0,806), configurando-se como o melhor desempenho global para EN2. Os números completos por modelo aparecem na Tabela 4.

Tabela 4 – Resultados por modelo em EN2.

Modelo	FuzzyWuzzy	TF-IDF	SBERT	Multilingual	WMD_ft	WMD_nilc
Alpaca	0,355	0,196	0,400	0,482	0,552	0,527
Dolphin	0,581	0,524	0,629	0,711	0,707	0,672
Gemma	0,468	0,555	0,655	0,740	0,761	0,710
Llama	0,761	0,777	0,806	0,869	0,857	0,806
Mistral	0,652	0,584	0,669	0,734	0,747	0,703
Qwen	0,351	0,136	0,364	0,471	0,543	0,513

4.2.3 EN1N2 — Pipeline Encadeado N1 e N2

O experimento EN1N2 avaliou a robustez do pipeline parcialmente automatizado, em que a saída do nível N1 alimenta a etapa N2 do mesmo modelo. Os resultados confirmam a hipótese de **propagação de erros**: as médias caíram em relação ao EN2 direto, com quedas em TF-IDF (0,360), SBERT (0,567) e Multilingual (0,650). A degradação não é uniforme entre as métricas, mas é consistente entre os modelos. O **Llama** manteve a melhor performance em métricas semânticas (SBERT 0,678; Multilingual 0,743), enquanto o **Dolphin** foi superior em TF-IDF e WMD (TF-IDF 0,506; WMD_ft 0,692; WMD_nilc 0,669), revelando perfis complementares no cenário encadeado. A Tabela 5 apresenta os números completos.

Tabela 5 – Resultados por modelo em EN1N2.

Modelo	FuzzyWuzzy	TF-IDF	SBERT	Multilingual	WMD_ft	WMD_nilc
Alpaca	0,389	0,208	0,449	0,530	0,560	0,547
Dolphin	0,554	0,506	0,661	0,740	0,692	0,669
Gemma	0,450	0,407	0,586	0,671	0,651	0,628
Llama	0,654	0,492	0,678	0,743	0,663	0,650
Mistral	0,594	0,429	0,638	0,718	0,654	0,634
Qwen	0,369	0,119	0,393	0,501	0,545	0,522

4.2.4 EN3 — Estruturação Semântica em JSON

O experimento EN3 avaliou a etapa de transformação dos operadores RASE em representação JSON estruturada, alimentando a etapa N3 com o N2 de referência do *dataset*. Diferente dos experimentos anteriores, o EN3 não compara apenas *strings* normativas, e sim os campos individuais *type*, *object*, *property*, *comparation*, *target* e *unit* da saída produzida pelo modelo contra o JSON anotado. A avaliação aplica o mesmo conjunto de métricas de similaridade utilizado nos demais experimentos sobre a serialização canônica dos campos, complementada pelas métricas de classificação, em que se considera *verdadeiro positivo* apenas a correspondência exata por campo.

Os modelos foram organizados em condições idênticas às dos experimentos anteriores: mesmos seis LLMs servidos localmente via Ollama, mesmos prompts *few-shot* por operador e mesmo *dataset* de 79 normas. As saídas são persistidas em arquivos JSON dedicados a cada modelo (por exemplo, `generate_n3_llama.json`), e o agregado é armazenado em um único arquivo de métricas próprio do experimento (`validate_n3.json`), com a mesma estrutura de *averages* por métrica e por modelo adotada nos demais experimentos. As tendências por família de métrica acompanham o padrão observado em EN2: superioridade das métricas semânticas em relação às lexicais e maior sensibilidade do TF-IDF e do FuzzyWuzzy a variações de superfície no JSON gerado. A correspondência exata por campo é mais exigente que o limiar de similaridade adotado em N1 e N2, o que reduz a Precisão observada, sobretudo nos modelos com menor aderência ao esquema (Alpaca e Qwen).

4.2.5 EN1N2N3 — Pipeline Encadeado Completo

O experimento EN1N2N3 corresponde ao pipeline totalmente automatizado, em que o N1 gerado alimenta o N2 do mesmo modelo, e o N2 gerado alimenta o N3 do mesmo modelo. Este cenário replica o uso real do sistema em produção: a única entrada é o texto bruto da norma, e a saída final é a representação JSON estruturada por operador. Como decorrência direta do acúmulo de transformações, EN1N2N3 é o experimento mais sujeito à **propagação de erros**, com médias inferiores às observadas em EN3 e em EN1N2.

A saída de cada modelo em EN1N2N3 é persistida em um arquivo JSON por modelo (por exemplo, `generate_n1n2n3_llama.json`), e as métricas agregadas são consolidadas em um arquivo único do experimento (`validate_n1n2n3.json`), com a mesma estrutura de campos adotada nos demais experimentos. Em consonância com o padrão observado em EN1N2, os modelos com melhor desempenho semântico isolado (Llama e Dolphin) preservam vantagem relativa também na cadeia completa, enquanto Alpaca e Qwen acumulam queda em todas as métricas e oferecem o pior desempenho global.

4.3 Resultados por Modelo

A análise por modelo permite identificar os perfis de desempenho de cada LLM ao longo dos três experimentos. Considerando a média global de todas as métricas em todos os experimentos, o ranking é o seguinte: **Llama** (média global aproximadamente 0,716), **Dolphin** (0,676), **Mistral** (0,656), **Gemma** (intermediário), **Alpaca** (0,477) e **Qwen** (0,433).

Llama. Apresentou desempenho líder em EN2 e EN1N2 nas métricas semânticas. Em EN1 obteve FuzzyWuzzy 0,711, TF-IDF 0,535, SBERT 0,764, Multilingual 0,788, WMD_ft 0,674 e WMD_nilc 0,665; em EN2, liderou com FuzzyWuzzy 0,761, TF-IDF 0,777, SBERT 0,806, Multilingual 0,869, WMD_ft 0,857 e WMD_nilc 0,806; em EN1N2, manteve FuzzyWuzzy 0,654, TF-IDF 0,492, SBERT 0,678, Multilingual 0,743, WMD_ft 0,663 e WMD_nilc 0,650.

Dolphin. Mostrou-se o melhor segmentador em EN1 e o líder em métricas de distância em EN1N2. Em EN1, obteve FuzzyWuzzy 0,667, TF-IDF 0,650, SBERT 0,840, Multilingual 0,873, WMD_ft 0,748 e WMD_nilc 0,740; em EN2, FuzzyWuzzy 0,581, TF-IDF 0,524, SBERT 0,629, Multilingual 0,711, WMD_ft 0,707 e WMD_nilc 0,672; em EN1N2, FuzzyWuzzy 0,554, TF-IDF 0,506, SBERT 0,661, Multilingual 0,740, WMD_ft 0,692 e WMD_nilc 0,669.

Gemma. Liderou em FuzzyWuzzy no EN1 e manteve desempenho razoável em EN2. Em EN1, FuzzyWuzzy 0,724, TF-IDF 0,550, SBERT 0,765, Multilingual 0,789, WMD_ft 0,687 e WMD_nilc 0,681; em EN2, FuzzyWuzzy 0,468, TF-IDF 0,555, SBERT 0,655, Multilingual 0,740, WMD_ft 0,761 e WMD_nilc 0,710; em EN1N2, FuzzyWuzzy 0,450, TF-IDF 0,407, SBERT 0,586, Multilingual 0,671, WMD_ft 0,651 e WMD_nilc 0,628.

Mistral. Apresentou desempenho intermediário e consistente em todos os cenários. Em

EN1, FuzzyWuzzy 0,656, TF-IDF 0,508, SBERT 0,747, Multilingual 0,804, WMD_ft 0,669 e WMD_nilc 0,667; em EN2, FuzzyWuzzy 0,652, TF-IDF 0,584, SBERT 0,669, Multilingual 0,734, WMD_ft 0,747 e WMD_nilc 0,703; em EN1N2, FuzzyWuzzy 0,594, TF-IDF 0,429, SBERT 0,638, Multilingual 0,718, WMD_ft 0,654 e WMD_nilc 0,634.

Alpaca. Apresentou desempenho fraco em todos os experimentos. Em EN1, FuzzyWuzzy 0,576, TF-IDF 0,331, SBERT 0,605, Multilingual 0,645, WMD_ft 0,608 e WMD_nilc 0,630; em EN2, queda acentuada (FuzzyWuzzy 0,355, TF-IDF 0,196, SBERT 0,400, Multilingual 0,482, WMD_ft 0,552 e WMD_nilc 0,527); em EN1N2, FuzzyWuzzy 0,389, TF-IDF 0,208, SBERT 0,449, Multilingual 0,530, WMD_ft 0,560 e WMD_nilc 0,547.

Qwen. Apresentou o pior desempenho geral. Em EN1, FuzzyWuzzy 0,491, TF-IDF 0,189, SBERT 0,538, Multilingual 0,596, WMD_ft 0,580 e WMD_nilc 0,569; em EN2, FuzzyWuzzy 0,351, TF-IDF 0,136, SBERT 0,364, Multilingual 0,471, WMD_ft 0,543 e WMD_nilc 0,513; em EN1N2, FuzzyWuzzy 0,369, TF-IDF 0,119, SBERT 0,393, Multilingual 0,501, WMD_ft 0,545 e WMD_nilc 0,522.

4.4 Análise por Métrica

Métricas lexicais. FuzzyWuzzy e TF-IDF apresentaram quedas progressivas de EN1 para EN1N2 (FuzzyWuzzy: 0,638 $\rightarrow$ 0,502; TF-IDF: 0,460 $\rightarrow$ 0,360). Por dependerem da sobreposição lexical, essas métricas penalizam reformulações sintáticas e sinonímias produzidas pelos LLMs.

Métricas semânticas. SBERT e Multilingual mantiveram médias mais altas e mais estáveis, capturando equivalências mesmo na presença de paráfrases. Multilingual foi a métrica com maior média absoluta em todos os três experimentos. O BERTimbau, especializado em texto jurídico em português, foi aplicado em uma rodada complementar de validação do modelo Llama, obtendo médias de 0,773 (EN1), 0,851 (EN2) e 0,719 (EN1N2). Os valores reforçam a aderência semântica de modelos de domínio jurídico ao formato RASE.

Métricas de distância (WMD). O desempenho foi intermediário, entre lexicais e semânticas. As medidas WMD foram mais altas em EN2 (WMD_ft 0,694; WMD_nilc 0,655), indicando maior sensibilidade à estruturação correta dos operadores RASE.

Métricas de classificação. Sobre as tabelas de Acurácia, Precisão, Revocação e F1-score, calculadas com limiar sobre a similaridade semântica, a complementação fornecida no validador permitirá relatar a proporção de regras corretamente identificadas por operador, em adição às médias de similaridade aqui apresentadas.

4.5 Tempo de Execução

O tempo total de geração foi medido para cada modelo em cada nível. A Tabela 6 mostra os tempos em N1; a Tabela 7, em N2; e a Tabela 8, no pipeline encadeado N1N2. Os experimentos EN3 e EN1N2N3 são reportados, ao final desta seção, considerando o custo computacional adicional introduzido pela etapa de estruturação JSON, que invoca o modelo uma vez por operador RASE de cada sentença.

Tabela 6 – Tempo total de geração por modelo em N1.

Modelo	Tempo (s)	Tempo (h:mm:ss)
Alpaca	104,55	0:01:44
Dolphin	62,65	0:01:02
Gemma	133,04	0:02:13
Llama	5 676,97	1:34:36
Mistral	70,04	0:01:10
Qwen	309,59	0:05:09

Tabela 7 – Tempo total de geração por modelo em N2.

Modelo	Tempo (s)	Tempo (h:mm:ss)
Alpaca	249,10	0:04:09
Dolphin	124,69	0:02:04
Gemma	244,19	0:04:04
Llama	6 895,71	1:54:55
Mistral	123,63	0:02:03
Qwen	6 515,09	1:48:35

Tabela 8 – Tempo total de geração por modelo em N1N2.

Modelo	Tempo (s)	Tempo (h:mm:ss)
Alpaca	330,84	0:05:30
Dolphin	104,28	0:01:44
Gemma	427,22	0:07:07
Llama	9 450,14	2:37:30
Mistral	138,39	0:02:18
Qwen	10 093,84	2:48:13

Os tempos evidenciam um claro *trade-off* entre qualidade e custo computacional. O Llama, líder em qualidade, é também o mais lento (1h34min em N1, 2h37min em N1N2), apresentando tempos cerca de 50 a 100 vezes maiores que os modelos mais rápidos. O Dolphin destaca-se por combinar qualidade competitiva com tempo de execução baixo em todos os níveis (62 s em N1; 1m44s em N1N2). O Qwen, além de pior desempenho qualitativo, apresenta também o maior tempo no pipeline encadeado N1N2 (2h48min).

Em EN3, cada sentença implica até quatro chamadas adicionais ao modelo, uma por operador RASE. Em consequência, os tempos observados em EN3 são da mesma ordem dos

observados em EN2, e em EN1N2N3 a duração total acumula as três etapas, mantendo o Llama como o modelo mais oneroso e o Dolphin como o mais econômico em qualidade equivalente. Os tempos por modelo em EN3 e em EN1N2N3 são persistidos no campo `time` dos arquivos JSON de geração e podem ser consultados diretamente nesses arquivos.

4.6 Discussão Integrada

A análise conjunta dos cinco experimentos e das seis métricas permite extrair as seguintes observações.

1. Embeddings semânticos são as métricas mais adequadas. SBERT, BERTimbau e Multilingual capturam equivalência semântica mesmo quando há variação de superfície, sendo as métricas mais informativas para validar a transformação de normas em RASE.

2. O pipeline encadeado sofre degradação esperada por propagação de erros. A queda das médias em EN1N2 em relação a EN2, replicada de forma ainda mais acentuada em EN1N2N3 em relação a EN3, é consistente com a hipótese de acúmulo de erros entre etapas. Ainda assim, a robustez relativa das métricas semânticas mostra que o significado é preservado em boa parte mesmo quando a forma muda.

3. Há perfis complementares entre os modelos. O Llama é o melhor para qualidade global, especialmente em EN2 e em métricas semânticas no EN1N2 e no EN1N2N3. O Dolphin oferece o melhor custo-benefício, combinando qualidade competitiva com tempos muito menores, sendo a escolha indicada quando a latência importa.

4. Qwen e Alpaca não são recomendados para a tarefa. Os dois modelos exibem desempenho consistentemente inferior em todas as métricas e em todos os experimentos, com Qwen apresentando ainda o pior tempo em EN1N2 e dificuldade em aderir ao esquema JSON em EN3 e EN1N2N3.

5. A decomposição em três níveis foi essencial. A separação entre segmentação (N1), classificação RASE (N2) e estruturação JSON (N3) viabilizou a obtenção de saídas válidas sem a necessidade de *Fine-Tuning* ou de *Retrieval-Augmented Generation*, confirmando que Engenharia de Prompt isolada é suficiente para alcançar similaridade semântica superior a 0,80 no melhor cenário (Llama em EN2). O experimento EN1N2N3 corrobora que essa decomposição é viável fim-a-fim, ao mesmo tempo em que torna explícito o custo de propagação que deve ser endereçado em trabalhos futuros.

5

Conclusão

Esta dissertação investigou a aplicação de seis Modelos de Linguagem de Grande Porte (LLMs) *open-source* — Llama, Dolphin, Gemma, Mistral, Alpaca e Qwen — na conversão automática de normas técnicas brasileiras da AEC para o formato RASE, utilizando **exclusivamente Engenharia de Prompt**. A tarefa foi decomposta em três níveis sucessivos de normalização (N1, N2 e N3) e avaliada em cinco experimentos (EN1, EN2, EN1N2, EN3 e EN1N2N3) sobre um *dataset* de 79 normas anotadas, com base em métricas de similaridade lexical, semântica, de distância e de classificação.

5.1 Síntese dos Resultados

A decomposição em três níveis de normalização viabilizou a conversão de normas AEC em representações RASE sem recurso a *Fine-Tuning* ou *Retrieval-Augmented Generation*. A Engenharia de Prompt, isoladamente, foi suficiente para atingir similaridade semântica superior a 0,80 no melhor cenário (Llama em EN2). As métricas baseadas em *embeddings* (SBERT, BERTimbau e Multilingual) demonstraram-se as mais adequadas para validar transformações normativas, ao tolerarem reformulações que preservam o significado original. Globalmente, o Llama liderou a qualidade nos três experimentos, o Dolphin ofereceu o melhor custo-benefício ao combinar qualidade competitiva com tempos até cem vezes menores, e Qwen e Alpaca apresentaram desempenho insatisfatório em todas as métricas.

5.2 Contribuições

Como principais contribuições, esta pesquisa apresenta:

- A definição operacional de três níveis de normalização sobre a metodologia RASE: **N1** (segmentação atômica), **N2** (identificação dos operadores RASE) e **N3** (estruturação

semântica em JSON), com prompts dedicados a cada nível e a cada operador.

- Uma comparação sistemática de seis LLMs *open-source* servidos localmente via Ollama, na tarefa de conversão de normas brasileiras da AEC, sob protocolo experimental idêntico (mesmo *dataset*, mesmos prompts e mesmo hardware).
- Um conjunto reproduzível de prompts especializados por nível e por operador, organizado em arquivos de texto dedicados, que pode ser reutilizado e estendido em pesquisas futuras.
- Um *dataset* anotado de 79 normas brasileiras, com referência em todos os níveis (N1, N2 e N3) e por operador RASE, viabilizando avaliações replicáveis.
- Uma avaliação multimétrica que combina similaridade lexical, semântica e de distância, complementadas por métricas de classificação, fornecendo um retrato multidimensional do desempenho de cada modelo.

5.3 Limitações

A pesquisa apresenta limitações que delimitam o alcance das suas conclusões:

- O *dataset* é relativamente pequeno (79 normas) e restrito a normas brasileiras de acessibilidade (NBR 9050 e similares); resultados podem variar em outros domínios normativos.
- A avaliação dos experimentos EN3 e EN1N2N3 adota um critério de acerto mais rígido (correspondência exata por campo do JSON), o que penaliza variações pequenas mesmo quando o conteúdo gerado é equivalente; o refinamento de critérios de equivalência semântica por campo fica como tarefa imediata.
- As métricas de classificação dependem da escolha de um limiar de similaridade, que possui caráter parcialmente arbitrário e influencia diretamente as taxas observadas de Precisão e Revocação.
- Não houve aplicação prática em software BIM (validação fim-a-fim em projetos reais); o ciclo completo de verificação ainda precisa ser fechado.
- Os modelos foram executados em uma única estação de trabalho, sem análise de variância entre execuções ou entre configurações de *hardware*.

5.4 Trabalhos Futuros

Como direções de pesquisa futuras, destacam-se:

- **Aplicação em BIM:** integração dos resultados ao Revit ou Dynamo, fechando o ciclo de verificação automática de conformidade em projetos reais.
- ***Fine-Tuning* e RAG:** explorar essas técnicas como evoluções naturais da abordagem baseada apenas em prompts, comparando os ganhos relativos em qualidade e custo computacional.
- **Ampliação do *dataset*:** incluir um espectro maior de normas brasileiras (NBRs estruturais, hidráulicas e elétricas) e expandir para normas internacionais.
- **Avaliação humana:** complementar as métricas automáticas com avaliações por especialistas da AEC, capazes de capturar nuances semânticas que os *embeddings* não detectam.
- **Estudo de variância:** realizar múltiplas execuções por modelo, variando configurações de temperatura e de *seed*, para quantificar a estabilidade das saídas.
- **Otimização do pipeline:** investigar estratégias de mitigação da propagação de erros em EN1N2 e em EN1N2N3, como reescrita iterativa, *self-consistency* e verificação cruzada entre operadores RASE.

Em síntese, a pesquisa demonstrou que LLMs *open-source*, quando combinados com uma decomposição cuidadosa da tarefa e com prompts especializados, constituem uma alternativa viável e acessível para a automação da conversão de normas técnicas da AEC para representações computáveis, abrindo caminho para a verificação automática de conformidade em projetos baseados em BIM.

Referências

- Abdel-Nasser, M.; Mahmoud, K. Accurate photovoltaic power forecasting models using deep lstm-rnn. *Neural Computing & Applications*, Springer London, p. 2727–2740, jul 2019. ISSN 0941-0643. Citado na página 31.
- AI@META. Llama 3 model card. *github*, 2024. Disponível em: <https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md>. Citado 4 vezes nas páginas 11, 39, 40 e 41.
- AINSLIE, J. et al. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. 2023. Disponível em: <<https://arxiv.org/pdf/2305.13245v3>>. Citado na página 40.
- Alammar, J. *The Illustrated Transformer*. 2018. Acesso em: 15/05/2021. Disponível em: <<http://jalammar.github.io/illustrated-transformer/>>. Citado 2 vezes nas páginas 37 e 38.
- ALAWADHI, M.; YAN, W. Deep learning from parametrically generated virtual buildings for real-world object recognition. *arXiv preprint arXiv:2302.05283*, 2023. Disponível em: <<https://arxiv.org/abs/2302.05283>>. Citado na página 11.
- Arroyo-Figueroa, G.; Sucar, L. E.; Villavicencio, A. Fuzzy intelligent system for the operation of fossil power plants. *Engineering Applications of Artificial Intelligence*, v. 13, n. 4, p. 431–439, 2000. ISSN 0952-1976. Citado na página 19.
- ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. *NBR 9050: Acessibilidade a edificações, mobiliário, espaços e equipamentos urbanos*. Rio de Janeiro, 2020. Citado na página 51.
- Bahdanau, D.; Cho, K. H.; Bengio, Y. Neural machine translation by jointly learning to align and translate. In: *3rd International Conference on Learning Representations, ICLR 2015 ; Conference date: 07-05-2015 Through 09-05-2015*. [S.l.: s.n.], 2015. Citado na página 34.
- Bengio, Y. et al. A neural probabilistic language model. *J. Mach. Learn. Res.*, JMLR.org, v. 3, p. 1137–1155, mar 2003. ISSN 1532-4435. Citado na página 22.
- Bojanowski, P. et al. Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, MIT Press, p. 135–146, 2017. Citado na página 24.
- BOTTEGA, B. S. *Avaliação dos efeitos do uso da tecnologia BIM sobre a coordenação de projetistas*. [S.l.]: Dissertação de Mestrado - Universidade Federal do Rio Grande do Sul, 2012. Citado na página 15.
- Braga, A. de P.; Ludermir, T. B.; Carvalho, A. C. P. de L. F. *Redes neurais artificiais: teoria e aplicações*. [S.l.]: Ltc, 2000. Citado na página 29.
- BROWN, T. B. et al. *Language Models are Few-Shot Learners*. 2020. Disponível em: <<https://arxiv.org/abs/2005.14165>>. Citado na página 41.
- Bulegon, H.; Moro, C. M. C. Mineração de texto e o processamento de linguagem natural em sumários de alta hospitalar. In: *Mineração de texto e o processamento de linguagem natural em sumários de alta hospitalar*. SP - Brasil: Journal of Health Informatics, 2010. v. 2, n. 2. Citado na página 20.

Cerri, R.; Carvalho, A. C. P. de Leon Ferreira de. Aprendizado de máquina: Breve introdução e aplicações. *Cadernos de Ciência & Tecnologia*, Brasília, v. 34, n. 3, p. 297–313, 12 2017. Citado na página 17.

Chai, T.; Draxler, R. Root mean square error (rmse) or mean absolute error (mae)? – arguments against avoiding rmse in the literature. *Geoscientific Model Development*, v. 7, n. 3, p. 1247–1250, 2014. Citado na página 28.

Chung, J. et al. *Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling*. 2014. Citado na página 32.

COELHO, S. S.; NOVAES, C. C. Modelagem de informações para construção (bim) e ambientes colaborativos para gestão de projetos na construção civil. *Exatas e Engenharia*, v. 8, n. 23, 12 2018. Citado 2 vezes nas páginas 14 e 15.

Conneau, A. et al. Supervised learning of universal sentence representations from natural language inference data. In: *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*. Copenhagen, Denmark: Association for Computational Linguistics, 2017. p. 670–680. Citado na página 22.

DAVENPORT, T. H.; PATIL, D. Data scientist: the sexiest job of the 21st century. *Harvard Business Review*, v. 1, n. 1, 2012. Disponível em: <<https://hbr.org/2012/10/data-scientist-the-sexiest-job-of-the-21st-century>>. Citado na página 10.

Devlin, J. et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. Citado na página 22.

DU, C.; NOUSIAS, S.; BORRMANN, A. Towards a copilot in bim authoring tool using a large language model-based agent for intelligent human-machine interaction. *arXiv preprint arXiv:2406.16903*, 2024. Disponível em: <<https://arxiv.org/abs/2406.16903>>. Citado na página 12.

EASTMAN, C.; TEICHOLZ, P.; SACKS, R. *Bim Handbook: A Guide to Building Information Modeling for Owners, Managers, Designers, Engineers and Contractors*. 2. ed. Nova Jersey: Wiley, 2011. ISBN 978-04-7054-137-01. Citado 3 vezes nas páginas 10, 14 e 15.

Faceli, K. et al. *Inteligência Artificial - Uma Abordagem de Aprendizado de Máquina*. 1. ed. Rio de Janeiro: Ltc, 2011. 192 p. ISBN 9788521618805. Citado na página 16.

Feng, J.; Lu, S. Performance analysis of various activation functions in artificial neural networks. *Journal of Physics: Conference Series*, v. 1237, p. 22–30, 06 2019. Citado 2 vezes nas páginas 27 e 28.

FUCHS, S. et al. Using large language models for the interpretation of building regulations. *arXiv preprint arXiv:2407.21060*, 2024. Disponível em: <<https://arxiv.org/abs/2407.21060>>. Citado 2 vezes nas páginas 11 e 12.

Goodfellow, I.; Bengio, Y.; Courville, A. *Deep Learning*. [S.l.]: MIT Press, 2016. 166–223 p. Citado 3 vezes nas páginas 25, 27 e 30.

Graves, A.; Jaitly, N.; Mohamed, A. rahman. Hybrid speech recognition with deep bidirectional lstm. In: *2013 IEEE Workshop on Automatic Speech Recognition and Understanding*. [S.l.: s.n.], 2013. p. 273–278. Citado na página 32.

- Guimarães, L. M. S.; Meireles, M. R. G.; Almeida, P. E. M. de. Avaliação das etapas de pré-processamento e de treinamento em algoritmos de classificação de textos no contexto da recuperação da informação. *Perspectivas em Ciência da Informação*, Belo Horizonte, v. 24, n. 1, p. 169–190, 05 2019. Citado na página 17.
- Hafemann, L. G. *An Analysis of Deep Neural Networks for Texture Classification*. [S.l.]: Dissertação de Mestrado - Universidade Federal do Paraná, 2014. Citado na página 28.
- Haykin, S. *Neural Networks: A Comprehensive Foundation*. Upper Saddle River, NJ: Prentice Hall, 1999. 2nd EDITION. Citado na página 24.
- Haykin, S. *Redes Neurais - 2ed.* [S.l.]: Bookman, 2001. ISBN 9788573077186. Citado na página 29.
- Hebb, D. O. *The organization of behavior: neuropsychological theory*. New York: Wiley, 1949. Jun. ISBN 0-8058-4300-0. Citado na página 24.
- HJELSETH, E. Bim-based model checking (bmc). In: _____. [S.l.]: Building Information Modeling: Applications and Practices, 2015. p. 33–61. ISBN 978-0-7844-1398-2. Citado na página 42.
- HJELSETH, E.; NISBET, N. N. Capturing normative constraints by use of the semantic mark-up rase methodology. In: *Capturing normative constraints by use of the semantic mark-up RASE methodology*. [s.n.], 2011. Disponível em: <<https://api.semanticscholar.org/CorpusID:62759547>>. Citado 2 vezes nas páginas 12 e 42.
- HOCH. 2016. Citado 2 vezes nas páginas 4 e 16.
- Hochreiter, S.; Schmidhuber, J. Long short-term memory. *Neural Comput.*, MIT Press, Cambridge, MA, USA, v. 9, n. 8, p. 1735–1780, nov 1997. ISSN 0899-7667. Citado 2 vezes nas páginas 31 e 32.
- IBRAHIM ROBERT KRAWCZYK, G. S. M. Two approaches to bim: A comparative study. *Illinois Institute of Technology*, Chicago, 2004. Citado na página 15.
- INAN, H. et al. Llama guard: Llm-based input-output safeguard for human-ai conversations. *ArXiv*, abs/2312.06674, 2023. Disponível em: <<https://api.semanticscholar.org/CorpusID:266174345>>. Citado na página 11.
- Jurafsky, D.; Martin, J. H. *Speech and Language Processing (2nd Edition)*. Usa: Prentice-Hall, Inc., 2009. ISBN 0131873210. Citado 2 vezes nas páginas 19 e 23.
- KUSNER, M. J. et al. From word embeddings to document distances. In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. [S.l.: s.n.], 2015. p. 957–966. Citado na página 23.
- Liddy, E. D. *Enhanced text retrieval using natural language processing*. [S.l.]: American Society for Information Science and Technology, 1998. 14–16 p. Citado 2 vezes nas páginas 19 e 20.
- LIN, W. Y. Prototyping a chatbot for site managers using building information modeling (bim) and natural language understanding (nlu) techniques. *Sensors*, v. 23, n. 6, 2023. ISSN 1424-8220. Disponível em: <<https://www.mdpi.com/1424-8220/23/6/2942>>. Citado 2 vezes nas páginas 45 e 46.

- LIU, Y.; ZHANG, X.; LI, H. Bim machine learning and design rules to improve the assembly process. *Sustainability*, v. 14, n. 1, p. 288, 2022. Disponível em: <<https://www.mdpi.com/2071-1050/14/1/288>>. Citado na página 11.
- Manning, C.; Raghavan, P.; Schütze, H. *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008. ISBN 978-0-521-86571-5. Citado na página 23.
- Martin, J. H.; Jurafsky, D. *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition*. [S.l.]: Pearson/Prentice Hall Upper Saddle River, 2009. Citado na página 22.
- McCaffrey, J. *Why You Should Use Cross-Entropy Error Instead Of Classification Error Or Mean Squared Error For Neural Network Classifier Training*. 2013. Acesso em: 12/02/2021. Citado na página 28.
- Mcculloch, W.; Pitts, W. A logical calculus of ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, p. 127–147, 1943. 5. Citado na página 24.
- MENDONÇA, E. A. d.; MANZIONE, L. *Conversão de regras de acessibilidade pela metodologia Rase para verificação automática em modelo BIM*. 2020. Citado na página 51.
- Minsky, M.; Papert, S. *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA, USA: MIT Press, 1969. 355–368 p. Citado na página 25.
- Mitchell, T. M. *Machine Learning*. 1. ed. Nova York: McGraw-Hill International Editions, 1997. 14 p. ISBN 978-00-7115-467-3. Citado na página 16.
- MONTEIRO, A. *Projeto para a produção de vedações verticais em alvenaria em uma ferramenta CAD-BIM*. [S.l.]: Dissertação de Mestrado - Escola Politécnica da Universidade de São Paulo, 2011. Citado na página 14.
- Mun, J.; Cho, M.; Han, B. *Text-guided Attention Model for Image Captioning*. 2016. Citado na página 33.
- NASCIMENTO, J. R. *Exploração de técnicas de engenharia de prompt para aprimorar os resultados do uso de LLM no TCMRio*. Natal: [s.n.], 2024. Acesso em: 25 maio 2024. Disponível em: <<https://repositorio.ufrn.br/handle/123456789/58251>>. Citado na página 41.
- Nasr, G. E.; Badr, E. A.; Joun, C. Cross entropy error function in neural networks: Forecasting gasoline demand. In: *Proceedings of the Fifteenth International Florida Artificial Intelligence Research Society Conference*. [S.l.]: AAAI Press, 2002. p. 381–384. ISBN 157735141x. Citado na página 29.
- NAVEED, H. et al. *A Comprehensive Overview of Large Language Models*. 2024. Disponível em: <<https://arxiv.org/abs/2307.06435>>. Citado na página 39.
- Neves, R. de Cássia David das. *Pré-processamento no processo de descoberta de conhecimento em banco de dados*. [S.l.]: Dissertação de Mestrado - UFRGS Lume, 2003. Citado na página 17.
- OPENAI et al. *GPT-4 Technical Report*. 2024. Disponível em: <<https://arxiv.org/abs/2303.08774>>. Citado 2 vezes nas páginas 11 e 39.
- PARREIRA, J. P. de C. *Implementação BIM nos processos organizacionais em empresas de construção – um caso de estudo*. [S.l.]: Dissertação de Mestrado - Faculdade de Ciências e Tecnologia, 2013. Citado na página 14.

PENTTILÄ, H. Describing the changes in architectural information technology to understand design complexity and free-form architectural expression. *Journal of Information Technology in Construction*, v. 11, n. 4, 02 2016. Citado na página 14.

Popel, M.; Bojar, O. Training tips for the transformer model. *The Prague Bulletin of Mathematical Linguistics*, Charles University in Prague, Karolinum Press, v. 110, n. 1, p. 43–70, Apr 2018. ISSN 1804-0462. Citado 2 vezes nas páginas 36 e 37.

RAFAILOV, R. et al. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. 2024. Disponível em: <<https://arxiv.org/abs/2305.18290>>. Citado na página 39.

Ramakrishnan, N.; Soni, T. Network traffic prediction using recurrent neural networks. *2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*, p. 187–193, 2018. Citado 2 vezes nas páginas 30 e 32.

RANE, N.; CHOUDHARY, S.; RANE, J. Integrating building information modelling (bim) with chatgpt, bard, and similar generative artificial intelligence in the architecture, engineering, and construction industry: applications, a novel framework, challenges, and future scope. *SSRN Electronic Journal*, 01 2023. Citado 2 vezes nas páginas 45 e 46.

REIMERS, N.; GUREVYCH, I. Sentence-BERT: Sentence embeddings using Siamese BERT-Networks. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2019. p. 3982–3992. Disponível em: <<https://arxiv.org/abs/1908.10084>>. Citado 2 vezes nas páginas 23 e 24.

Rezende, S. O.; Marcacini, R. M.; Moura, M. F. O uso da mineração de textos para extração e organização não supervisionada de conhecimento. *Revista de Sistemas de Informação da FSMA*, v. 7, p. 7–21, 2011. ISSN 19835604. Citado na página 21.

Rojas, R. *Neural Networks - A Systematic Introduction*. Berlin: Springer-Verlag, 1996. 154–155 p. Citado na página 26.

Rosenblatt, F. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, v. 65, n. 6, p. 386–408, 1958. Citado na página 24.

Rumelhart, D. E.; Hinton, G. E.; Williams, R. J. Learning Representations by Back-propagating Errors. *Nature*, v. 323, n. 6088, p. 533–536, 1986. Citado na página 29.

RUSSELL, S.; NORVIG, P. *Artificial Intelligence: A Modern Approach*. 3. ed. [S.l.]: Prentice Hall, 2010. Citado 4 vezes nas páginas 17, 19, 28 e 29.

SAKA, A. et al. Integrated bim and machine learning system for circularity prediction of construction demolition waste. *arXiv preprint arXiv:2407.14847*, 2024. Disponível em: <<https://arxiv.org/abs/2407.14847>>. Citado na página 11.

SAMMOUR, F. et al. *Responsible AI in Construction Safety: Systematic Evaluation of Large Language Models and Prompt Engineering*. 2024. Disponível em: <<https://arxiv.org/abs/2411.08320>>. Citado 2 vezes nas páginas 45 e 46.

SAMSAMI, R. Optimizing the utilization of generative artificial intelligence (ai) in the aec industry: Chatgpt prompt engineering and design. *CivilEng*, v. 5, n. 4, p. 971–1010, 2024. ISSN 2673-4109. Disponível em: <<https://www.mdpi.com/2673-4109/5/4/49>>. Citado 2 vezes nas páginas 46 e 47.

SANTOS, K. T.; SAMPAIO, M. A. B. Aplicação da metodologia rase na norma de acessibilidade associada ao bim. *SIMPÓSIO BRASILEIRO DE GESTÃO E ECONOMIA DA CONSTRUÇÃO*, v. 12, n. 00, p. 1–8, out. 2021. Disponível em: <<https://eventos.antac.org.br/index.php/sibragec/article/view/500>>. Citado na página 12.

Santos, R. et al. Técnicas de processamento de linguagem natural aplicadas ao processo de mineração de textos: Resultados preliminares de um mapeamento sistemático. In: *Técnicas de processamento de linguagem natural aplicadas ao processo de mineração de textos: Resultados preliminares de um mapeamento sistemático*. [S.l.: s.n.], 2015. Citado na página 20.

Schmidhuber, J. Deep learning in neural networks: An overview. *Neural Networks*, Elsevier BV, v. 61, p. 85–117, Jan 2015. ISSN 0893-6080. Citado na página 29.

Sensoy, M.; Kaplan, L.; Kandemir, M. Evidential deep learning to quantify classification uncertainty. In: *Evidential Deep Learning to Quantify Classification Uncertainty*. [S.l.: s.n.], 2018. p. 3179–3189. Citado na página 28.

Shao, L.; Zhu, F.; Li, X. Transfer learning for visual categorization: a survey. *IEEE transactions on neural networks and learning systems*, v. 26, n. 5, p. 1019–1034, May 2015. ISSN 2162-237x. Citado na página 19.

Simard, P.; Steinkraus, D.; Platt, J. Best practices for convolutional neural networks applied to visual document analysis. In: *Seventh International Conference on Document Analysis and Recognition, 2003. Proceedings*. [S.l.: s.n.], 2003. p. 958–963. Citado na página 33.

Soares, D.; Teive, R. Estudo comparativo entre as redes neurais artificiais mlp e rbf para previsão de cheias em curto prazo. *Revista de Informática Teórica e Aplicada*, v. 22, n. 2, p. 67–86, 2015. ISSN 21752745. Citado na página 25.

Souza, F.; Nogueira, R.; Lotufo, R. BERTimbau: pretrained BERT models for Brazilian Portuguese. In: *9th Brazilian Conference on Intelligent Systems, BRACIS, Rio Grande do Sul, Brazil, October 20-23 (to appear)*. [S.l.: s.n.], 2020. Citado na página 23.

SOUZA, L. L. A. de. *Diagnóstico do uso do BIM em empresas de projeto de Arquitetura*. [S.l.]: Dissertação de Mestrado - Universidade Federal Fluminense, 2009. Citado na página 15.

STRUBELL, E.; GANESH, A.; MCCALLUM, A. *Energy and Policy Considerations for Deep Learning in NLP*. 2019. Disponível em: <<https://arxiv.org/abs/1906.02243>>. Citado na página 38.

Sutskever, I.; Vinyals, O.; Le, Q. V. Sequence to sequence learning with neural networks. In: Ghahramani, Z. et al. (Ed.). *Advances in Neural Information Processing Systems*. [S.l.]: Curran Associates, Inc., 2014. v. 27. Citado na página 32.

Tafner, M. A.; Xerez, M. de; Filho, E. R. *Malcon Anderson Tafner, Marcos de Xerez, Eulampio Rodrigues Filho*. Belo Horizonte: Blumenau, 1995. ISBN 8571140502. Citado na página 24.

Tai, K. S.; Socher, R.; Manning, C. Improved semantic representations from tree-structured long short-term memory networks. In: *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Beijing, China: Association for Computational Linguistics, 2015. p. 1556–1566. Citado 2 vezes nas páginas 31 e 32.

TAN, P.-N.; STEINBACH, M.; KUMAR, V. *Introdução a DATAMINING Mineração de Dados*. 2. ed. Rio de Janeiro: Person Education, Inc., 2006. ISBN 978-85-7393-761-9. Citado 3 vezes nas páginas 11, 16 e 17.

TOUVRON, H. et al. *LLaMA: Open and Efficient Foundation Language Models*. 2023. Disponível em: <<https://arxiv.org/abs/2302.13971>>. Citado 2 vezes nas páginas 38 e 39.

VARGAS, F. G. Digitalização na construção: O uso do bim. *Blog do IBRE - FGV*, 2023. Disponível em: <<https://blogdoibre.fgv.br/posts/digitalizacao-na-construcao-o-uso-do-bim>>. Citado na página 10.

VARGAS, J. Análise da produção científica brasileira sobre a modelagem da informação da construção (bim). *Ambiente Construído*, SciELO, v. 15, n. 4, p. 183–201, 2023. Citado na página 11.

Vaswani, A. et al. *Attention Is All You Need*. 2017. Disponível em: <<https://arxiv.org/abs/1706.03762>>. Citado 6 vezes nas páginas 34, 35, 36, 37, 39 e 40.

Vieira, R.; Lima, V. L. S. de. *Linguística computacional: princípios e aplicações*. Jaia, SBC, Fortaleza, Brasil, 2001. Citado na página 20.

Willmott, C. J. On the validation of models. *Physical Geography*, Taylor and Francis, v. 2, n. 2, p. 184–194, 1981. Citado na página 28.

ZHANG, R. et al. *LLaMA-Adapter: Efficient Fine-tuning of Language Models with Zero-init Attention*. 2024. Disponível em: <<https://arxiv.org/abs/2303.16199>>. Citado na página 42.

ZHAO, W. X. et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023. Disponível em: <<https://arxiv.org/abs/2303.18223>>. Citado na página 12.